

CSE 753: Advanced Reinforcement Learning

Contents

1	Policy Gradient Methods	2
2	Actor Critic Method	23
3	Imitation Learning	46

1 Policy Gradient Methods

1.1 Generalized Policy Iteration

Generalized Policy Iteration — the core idea behind how an AI agent learns to make better decisions over time.

The Big Idea

The agent follows a simple two-step loop, repeated over and over:

1. **Collect experience** — Run the current policy (the agent’s current strategy, called π) out in the world and record what happened. These recorded sequences of actions and outcomes are called **trajectories** — basically a replay of what the agent did from start to finish.
2. **Improve the policy** — Look at all those trajectories. Which ones led to good outcomes? Which led to bad ones? Use that information to update the policy so it’s slightly smarter than before.

Then repeat. That’s it. The whole loop.

What “Initialize” Means

Before the loop starts, you need to give the agent some starting policy. You have three options:

- **Random** — just pick actions randomly at first (agent knows nothing)
- **Imitation learning** — copy how an expert does it
- **Heuristics** — use simple hand-coded rules as a starting point

How It Improves Over Time

The diagram on the right shows three snapshots — Iteration 1, 50, and 100. Think of each curved line as one trajectory (one attempt the agent made from start to finish).

- Red X = bad outcome (low reward)
- Yellow check = okay outcome
- Green check = good outcome

Stage	What’s happening
Early (Iter 1)	Most attempts fail. The agent is basically guessing. Lots of exploration.
Mid (Iter 50)	Getting better. Some trajectories succeed. Fewer failures.
Late (Iter 100)	Almost all trajectories lead to good outcomes. The policy has converged.

The Key Takeaway

The agent doesn’t need to know the rules of the world in advance. It just tries things, sees what worked, updates itself, and tries again. Over hundreds of iterations, this simple loop produces a very good policy.

1.2 Online Learning

Is this online learning only?

No. The slide specifically describes a **batch** update style — it says “collect a **batch** of data (trajectories)”, then update. That’s not online learning.

Here’s the distinction:

- **Online learning** — update the policy after **every single step** (or every transition). You don’t wait.
- **Batch learning** — collect a **full set of trajectories** first, then update the policy once using all of them. That’s what the slide shows.
- **Mini-batch** — a middle ground; collect a fixed number of episodes, update, repeat.

GPI as a **general framework** doesn’t lock you into one style. It just says “evaluate → improve → repeat.” How often you update is an implementation choice. That’s why both REINFORCE (batch, full episodes) and Actor-Critic methods (online or near-online, step-by-step) are both considered instances of GPI.

What happens in online learning?

In online learning, the agent updates the policy **after every single step** — no waiting, no batching.

The loop is:

1. Take one action in the environment
2. See the result (reward + next state)
3. Immediately update the policy
4. Take the next action using the already-updated policy

So the policy is changing **continuously** as the agent moves through the world. By the time the episode ends, the policy has already been updated dozens or hundreds of times.

The tradeoff: updates are fast and frequent, but each update is noisy because it’s based on just one data point instead of a whole batch of experience.

In online learning agent interacts with real environment directly. Right?

Not necessarily. “Online learning” just means **updating after every step** — it says nothing about whether the environment is real or simulated.

- In a **real environment** — yes, the agent takes a real action, gets a real reward, updates immediately.
- In a **simulated environment** — the agent still updates after every step, it’s still online learning. The environment just happens to be a simulator.

The word **online** refers to the **frequency of updates**, not the **type of environment**.

The distinction you might be thinking of is **Online (real environment)** vs. **Offline / Batch RL (replay buffer or pre-collected dataset)** — here “offline” means the

agent learns from a fixed dataset with no new interactions at all. But that’s a different axis than simulated vs. real.

1.3 Policy Without Q Values

Page 3 is titled “**Policy without Q values**” and introduces a new way of thinking about policies.

The Setup

So far in the course, GPI-based algorithms figured out a good policy by first computing $Q(s, a)$ — the expected reward of taking action a in state s . Once you know Q values well enough, you derive the policy from them (e.g., pick the action with the highest Q value). The slide says: what if we skip Q values entirely and just **directly build the policy itself**?

The New Idea: Parameterized Policy

Instead of storing Q values and deriving the policy from them, we represent the policy as a **mathematical function with adjustable knobs**. Those knobs are called **parameters**. We then tune the knobs directly until the policy gets good. This is called **parameterization** — you describe the policy using a set of numbers (the parameters), and learning means finding the right numbers.

The Equation

$$\pi(a \mid s, \vec{\theta}) = \Pr\{A_t = a \mid S_t = s, \vec{\theta}_t = \vec{\theta}\}$$

Symbol glossary:

Symbol	Meaning
--------	---------

π	The policy — the agent’s decision-making rule
a	A specific action the agent could take
s	The current state the agent is in
$\vec{\theta}$	The parameter vector — a list of numbers that defines the shape of the policy
d'	The number of parameters in $\vec{\theta}$ (its size/dimension)
$\Pr\{\cdot\}$	Probability of something happening
A_t	The action actually taken at time step t
S_t	The state the agent was in at time step t
t	A specific moment in time during the episode

The policy $\pi(a \mid s, \vec{\theta})$ gives you the **probability of choosing action** a , given that you’re in state s and your policy’s parameters are currently set to $\vec{\theta}$. The policy is no longer a lookup table — it’s a function. You plug in the state and the parameters, and it spits out a probability for each possible action. Learning now means **adjusting** $\vec{\theta}$ to make good actions more probable and bad actions less probable.

1.4 Parameterizing Policy

Page 4 is titled “**Parameterizing Policy**” and answers a practical question: if the policy is just a function with parameters, what rules must it follow, and what’s a concrete

example of such a function?

Two Rules Every Policy Must Obey

Since $\pi(a \mid s, \theta)$ outputs a **probability**, it has to follow the basic laws of probability. The slide gives two constraints.

Rule 1

$$\pi(a \mid s, \theta) \geq 0 \quad \text{for all } a \in \mathcal{A} \text{ and } s \in \mathcal{S}$$

Symbol glossary:

Symbol	Meaning
--------	---------

$\pi(a \mid s, \theta)$	Probability of taking action a in state s with parameters θ
$\mathcal{A}$	The set of all possible actions
$\mathcal{S}$	The set of all possible states

Every probability the policy outputs must be zero or positive. You can't have a negative probability — that makes no sense. This is just the most basic rule of probability.

Rule 2

$$\sum_{a \in \mathcal{A}} \pi(a \mid s, \theta) = 1 \quad \text{for all } s \in \mathcal{S}$$

Symbol glossary:

Symbol	Meaning
--------	---------

$\sum_{a \in \mathcal{A}}$	Add up over every possible action in $\mathcal{A}$
----------------------------	--

If you add up the probabilities of all possible actions in any given state, the total must equal 1. The agent has to do *something* — so all options together must account for 100% of the probability.

The Softmax Function — A Concrete Example

$$\pi(a \mid s, \theta) = \frac{e^{h(s,a,\theta)}}{\sum_{b \in \mathcal{A}} e^{h(s,b,\theta)}}$$

Symbol glossary:

Symbol	Meaning
--------	---------

$h(s, a, \theta)$	The preference score for action a in state s — a raw number produced by the parameter
$e^{(\cdot)}$	The exponential function — turns any number into a positive value, amplifies differences
b	A dummy variable representing each action in the sum (used to loop over all actions)
$\sum_{b \in \mathcal{A}} e^{h(s,b,\theta)}$	Total of exponential scores across all actions — used to normalize

First, compute a preference score $h(s, a, \theta)$ for every action. Then take e to the power of each score to make them all positive. Finally, divide each action's score by the total of all scores, so everything adds up to 1. The result is a valid probability for each action. Actions with higher preference scores get higher probabilities, but no action is ever completely ignored (its probability never hits exactly zero).

The Greedy Behavior Note

If the preference $h(s, a, \theta)$ of a particular action grows very large, the policy tends to act greedily. This means: if one action's score gets much bigger than all others, the softmax will assign it nearly 100% probability and ignore the rest. The policy stops exploring and just always picks that one action — it becomes **greedy**.

1.5 Preference Score and Feature Vectors

Who defines the preference score? How is it defined?

The parameters θ define it — specifically, $h(s, a, \theta)$ is a function that you design, and θ are the learnable weights inside that function.

How it's defined in practice

The most common choice is a **linear function**:

$$h(s, a, \theta) = \theta^\top x(s, a)$$

where $x(s, a)$ is a feature vector you hand-craft to describe the state-action pair, and θ is a weight vector the algorithm learns. The dot product gives a single score.

A more powerful choice is a **neural network** — you feed the state s in, and the network outputs one score per action. The weights of the network are θ , and they get updated through gradient descent.

So who decides the structure?

- **You (the designer)** choose the form of h — linear, neural network, etc.
- **The learning algorithm** (e.g., REINFORCE) then adjusts θ automatically over training to make good actions score higher and bad actions score lower.

The preference score has no inherent meaning on its own — it's just a raw number. What matters is the **relative difference** between scores across actions, because softmax only cares about which score is bigger, not the absolute values.

“ $x(s, a)$ is a feature vector you hand-craft”. What do you mean by hand-craft?

It means **you manually decide** what information to extract from the state-action pair and feed into the function.

Example — a robot navigating a grid

Say the state s is the robot's position, and action a is “move right.” You might hand-craft the feature vector as:

$$x(s, a) = \begin{bmatrix} \text{distance to goal} \\ \text{distance to wall} \\ 1 \text{ if action} = \text{right, else } 0 \\ 1 \text{ if action leads to open space, else } 0 \end{bmatrix}$$

You — the human — decided that these four pieces of information matter. You wrote that logic manually. That’s what “hand-crafted” means.

Why is this a problem?

- You might miss important features
- You might include useless ones
- For complex problems (images, text, games), it’s nearly impossible to hand-craft good features

That’s exactly why neural networks replaced hand-crafted features

A neural network takes the raw state (e.g., raw pixels) and **learns its own internal features automatically** from experience. You don’t decide what matters — the network figures it out. The weights θ inside the network serve the same role as the hand-crafted $x(s, a)$, but nothing is manually designed. So hand-crafting was the old way. Neural networks are the modern way.

1.6 The RL Objective

Page 6 is titled “**The RL Objective**” — it formally states what the agent is actually trying to maximize, and how to estimate it in practice.

The Big Goal

The agent wants to find the best possible parameters θ for its policy. “Best” means: the parameters that produce the highest total reward on average across all possible trajectories.

Equation 1 — Finding the Best Parameters

$$\vec{\theta}^* = \arg \max_{\theta} \mathbb{E}_{\tau \sim p_{\theta}(\tau)} [r(\tau)]$$

Symbol glossary:

Symbol	Meaning
$\vec{\theta}^*$	The optimal (best) parameter vector — what we’re searching for
$\arg \max_{\theta}$	“Give me the value of θ that makes the thing inside as large as possible”
τ	A trajectory — one full sequence of states and actions from start to finish
$p_{\theta}(\tau)$	The probability of trajectory τ happening under policy θ
$\mathbb{E}_{\tau \sim p_{\theta}(\tau)}[\cdot]$	The average value of something, taken over all trajectories the policy generates
$r(\tau)$	The total reward collected along trajectory τ

We want the parameters θ^* that make the **average total reward across all trajectories** as high as possible. We're not optimizing for one lucky run — we want good performance on average.

Equation 2 — Expanding the Reward

$$= \arg \max_{\theta} \mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[\underbrace{\sum_t r(s_t, a_t)}_{J(\theta)} \right]$$

Symbol glossary:

Symbol Meaning

$\sum_t$	Add up over every time step t in the trajectory
$r(s_t, a_t)$	The reward received at time step t , for being in state s_t and taking action a_t
$J(\theta)$	The objective function — the single number we want to maximize; our score for how good

The total reward of a trajectory $r(\tau)$ is just the sum of all the small rewards collected at each individual step. $J(\theta)$ is the name we give to the expected value of that sum — it's our score for how good a given θ is.

Equation 3 — How to Actually Compute It

$$J(\theta) \approx \frac{1}{N} \sum_i \sum_t r(s_{i,t}, a_{i,t})$$

Symbol glossary:

Symbol Meaning

$\approx$	Approximately equal to
N	Number of trajectories (sample runs) collected
$\sum_i$	Add up over each trajectory i (each run)
$\sum_t$	Add up over each time step within that trajectory
$r(s_{i,t}, a_{i,t})$	Reward at time step t in trajectory i
$\frac{1}{N}$	Divide by number of trajectories to get the average

You can't compute the true expectation $J(\theta)$ exactly — that would require running infinitely many trajectories. Instead, you run N trajectories, add up all the rewards collected across all time steps in all trajectories, and divide by N to get the average. This average is a good enough approximation of the true $J(\theta)$.

The Diagram on the Right

This is exactly what the approximation looks like in practice. The dot on the left is the starting state. Each curved arrow is one trajectory — one attempt by the agent. Some end in red X (bad outcome, low reward), some in yellow or green checkmark (good or great outcome). You run N of these, average the total rewards, and that gives you your estimate of $J(\theta)$.

1.7 Pages 7–8: Deriving the Gradient

Page 7 — Gradient of the RL Objective

Quick recap. Remember from page 6: the agent’s goal is to maximize $J(\theta)$, the average total reward it gets by following its policy. To improve the policy, we need to compute the **gradient** of $J(\theta)$ — basically, the direction in which to nudge the policy knobs (θ) to get more reward.

Equation 1 — Rewriting the objective as an integral.

$$J(\theta) = E_{\tau \sim p_\theta(\tau)} \left[\sum_t r(s_t, a_t) \right] = \int p_\theta(\tau) r(\tau) d\tau$$

Symbol	Meaning
--------	---------

$J(\theta)$	The score of the policy — total expected reward
-------------	---

θ	The policy’s knobs (parameters)
----------	---------------------------------

τ	A trajectory — a full playthrough: $s_1, a_1, s_2, a_2, \dots$
--------	--

$p_\theta(\tau)$	Probability of that trajectory happening under policy θ
------------------	--

$r(\tau)$	Total reward collected along that trajectory
-----------	--

$\int \dots d\tau$	“Sum over all possible trajectories”
--------------------	--------------------------------------

This is just rewriting the expected reward in math notation. Instead of saying “average reward over many runs,” we write it as an integral — summing $p_\theta(\tau) \cdot r(\tau)$ over every possible trajectory. The integral is mathematically equivalent to the expectation, which sets us up for taking the gradient.

Equation 2 — Taking the gradient (the log-derivative trick).

$$\nabla_\theta J(\theta) = \int \nabla_\theta p_\theta(\tau) r(\tau) d\tau = \int p_\theta(\tau) \nabla_\theta \log p_\theta(\tau) r(\tau) d\tau = E_{\tau \sim p_\theta} [\nabla_\theta \log p_\theta(\tau) r(\tau)]$$

Symbol	Meaning
--------	---------

∇_θ	“How much does this change when we tweak θ ?” (the gradient)
-----------------	---

$\nabla_\theta p_\theta(\tau)$	Gradient of the trajectory probability — hard to work with directly
--------------------------------	---

$\nabla_\theta \log p_\theta(\tau)$	Gradient of the <i>log</i> of the trajectory probability — much easier
-------------------------------------	--

We want the gradient of $J(\theta)$, but differentiating $p_\theta(\tau)$ directly is messy because it involves the unknown environment dynamics. The **log-derivative trick** says: $p_\theta(\tau) \cdot \nabla_\theta \log p_\theta(\tau) = \nabla_\theta p_\theta(\tau)$. This lets us swap the hard term out and rewrite the whole gradient as an **expectation** — which means we can estimate it just by running the policy and averaging, no integrals needed.

The key identity.

$$p_\theta(\tau) \nabla_\theta \log p_\theta(\tau) = p_\theta(\tau) \cdot \frac{\nabla_\theta p_\theta(\tau)}{p_\theta(\tau)} = \nabla_\theta p_\theta(\tau)$$

This is just the chain rule of calculus: the derivative of $\log(x)$ is $\frac{1}{x}$. Multiplying $p_\theta(\tau)$ on both sides cancels it out, proving the substitution above is valid.

Page 8 — Expanding the Gradient into Something Computable

Now the slide answers: “Okay, but how do we actually **compute** $\nabla_\theta \log p_\theta(\tau)$?”

Equation 3 — What is $p_\theta(\tau)$?

$$p_\theta(\tau) = p(s_1) \prod_{t=1}^T \pi_\theta(a_t | s_t) p(s_{t+1} | s_t, a_t)$$

Symbol	Meaning
$p(s_1)$	Probability of starting in state s_1 (set by the environment, not the agent)
$\prod_{t=1}^T$	Multiply together across all time steps $t = 1$ to T
$\pi_\theta(a_t s_t)$	Policy’s probability of picking action a_t when in state s_t
$p(s_{t+1} s_t, a_t)$	Environment’s probability of moving to s_{t+1} given state and action

A trajectory’s probability is the product of every decision and every world transition along the way. Think of it like the probability of a specific path through a maze — it’s the starting probability times the chance you took each turn times the chance the world responded the way it did, multiplied across every step.

Equation 4 — Expanding $\log p_\theta(\tau)$.

$$\nabla_\theta \left[\log p(s_1) + \sum_t \log \pi_\theta(a_t | s_t) + \log p(s_{t+1} | s_t, a_t) \right]$$

Taking the log of a product turns it into a sum (since $\log(ABC) = \log A + \log B + \log C$). The critical insight is that $\log p(s_1)$ and $\log p(s_{t+1} | s_t, a_t)$ do not contain θ at all — they are controlled by the environment, not the policy. So their gradients are **zero** and they vanish. Only $\log \pi_\theta(a_t | s_t)$ survives.

Equation 5 — The practical gradient formula.

$$\nabla_\theta J(\theta) = E_{\tau \sim p_\theta} \left[\left(\sum_{t=1}^T \nabla_\theta \log \pi_\theta(a_t | s_t) \right) \left(\sum_{t=1}^T r(s_t, a_t) \right) \right]$$

Symbol	Meaning
$\sum_{t=1}^T \nabla_\theta \log \pi_\theta(a_t s_t)$	How to nudge the policy to make each action more or less likely
$\sum_{t=1}^T r(s_t, a_t)$	Total reward collected during the trajectory

The gradient is the product of two sums — “how much did each action’s probability change when we tweak θ ?” multiplied by “how much total reward did we get?” If a trajectory got high reward, we push θ in the direction that made those actions more

likely. If it got low reward, we push the other way. Crucially, the environment dynamics $p(s_{t+1} \mid s_t, a_t)$ are **completely gone** — we only need the policy and the rewards.

Equation 6 — The sample estimate (how we actually compute it).

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_i \left(\sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(a_{i,t} \mid s_{i,t}) \right) \left(\sum_{t=1}^T r(s_{i,t}, a_{i,t}) \right)$$

Symbol	Meaning
N	Number of sampled trajectories (rollouts)
i	Index for each sampled trajectory
$\frac{1}{N} \sum_i$	Average over all N rollouts
$a_{i,t}, s_{i,t}$	Action and state at time t in the i -th trajectory

Since we cannot compute an expectation over *all* possible trajectories, we approximate it by running the policy N times, computing the gradient formula for each run, and averaging. This is the same idea as estimating the average of a die roll by rolling it many times — the more rollouts, the better the estimate. This is the actual algorithm you would implement in code.

1.8 Page 9: The Full REINFORCE Algorithm

This page puts everything from pages 7 and 8 together into one complete, runnable algorithm. It has a name: **REINFORCE**, also called the **vanilla policy gradient**. “Vanilla” just means the plain, basic version — no fancy extras yet.

Think of it like a recipe with 4 steps that you repeat over and over until the agent gets good.

Step 1 — Collect experience.

Sample $\{\tau^i\}$ from $\pi_{\theta}(a_t \mid s_t)$

Symbol	Meaning
$\{\tau^i\}$	A collection of trajectories (multiple full playthroughs), indexed by i
$\pi_{\theta}(a_t \mid s_t)$	The current policy — given state s_t , it picks action a_t

Run your current policy in the environment multiple times and record what happened each time — which states you visited, which actions you took, and what rewards you got. This is just the agent “playing the game” several times and taking notes.

Step 2 — Compute the gradient.

$$\nabla_{\theta} J(\theta) \approx \sum_i \left(\sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(a_t^i \mid s_t^i) \right) \left(\sum_{t=1}^T r(s_t^i, a_t^i) \right)$$

Symbol	Meaning
$\nabla_{\theta} J(\theta)$	The gradient — which direction to push the policy knobs
$\sum_i$	Sum (add up) over all collected trajectories
$\nabla_{\theta} \log \pi_{\theta}(a_t^i s_t^i)$	How much action a_t in trajectory i would change if we tweaked θ
$\sum_{t=1}^T r(s_t^i, a_t^i)$	Total reward collected in trajectory i

For each recorded trajectory, multiply “how sensitive was the policy to its own choices” by “how much reward did it get?” Add this up across all trajectories. Trajectories with high reward push the gradient hard in a direction that makes those actions more likely; trajectories with low reward push in the opposite direction. This is the signal that tells us how to improve.

Step 3 — Update the policy.

$$\vec{\theta} = \vec{\theta} + \alpha \nabla_{\theta} J(\theta)$$

Symbol	Meaning
$\vec{\theta}$	The policy’s parameters (knobs) — a vector of numbers
α	Learning rate — controls how big a step to take
$\nabla_{\theta} J(\theta)$	The gradient computed in Step 2

Nudge the policy parameters θ in the direction of the gradient by a small amount controlled by α . Note it is a **plus** sign, not minus — because we are doing **gradient ascent** (climbing toward higher reward), unlike regular gradient descent used in supervised learning which minimizes a loss. A large α means big jumps (risky); a small α means careful, slow improvement.

Step 4 — Repeat until convergence.

Loop back to Step 1 with the newly updated policy. Collect new data, compute a new gradient, update again. Keep going until the policy stops improving noticeably — that is convergence.

The big picture. The diagram on the slide shows this loop: collect experience by running the policy, then improve the policy using that data, then repeat. REINFORCE looks at all collected trajectories, figures out which actions led to good outcomes, and makes those actions more likely next time. It is literally trial and error, formalized into math.

1.9 Page 10: Understanding What the Gradient Actually Does

This page is not introducing new math. It is stepping back and asking: **what does this gradient formula actually mean in plain English?** It is the intuition behind everything derived on pages 7–9.

The gradient (same formula, new lens).

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_i \left(\sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(a_t^i | s_t^i) \right) \left(\sum_{t=1}^T r(s_t^i, a_t^i) \right)$$

Symbol	Meaning
$\frac{1}{N} \sum_i$	Average over all N collected trajectories
$\nabla_{\theta} \log \pi_{\theta}(a_t^i s_t^i)$	Direction to push θ to make action a_t in trajectory i more likely
$\sum_{t=1}^T r(s_t^i, a_t^i)$	Total reward of trajectory i — the weight

The total reward of a trajectory acts as a **multiplier** on the gradient. If a trajectory earned high reward, its gradient contribution gets multiplied by a big positive number — meaning those actions get strongly encouraged. If the reward was negative, the multiplier flips the sign and those actions get discouraged. The gradient is literally a weighted vote from every trajectory.

The three key points from the slide.

1. **Increase the likelihood of actions you took in high-reward trajectories.** If a playthrough went well, assume the actions you took during it were probably good. Make those actions more likely in the future.
2. **Decrease the likelihood of actions you took in negative-reward trajectories.** If a playthrough went badly, assume those actions were probably bad. Make them less likely going forward.
3. **Do more of the good stuff, less of the bad stuff.** That is the whole algorithm in one sentence. The math just makes this idea precise and automatic.

The diagram. The picture shows several trajectories starting from the same point. Some end in failure and one ends in success. After running REINFORCE, the policy will shift probability mass toward whatever sequence of actions led to the successful path, and away from the actions that led to failure. Over many iterations, the successful path becomes more and more likely.

Why this page matters. Everything on pages 7 and 8 was dense derivation. This page is the payoff — it tells you that all that math ultimately boils down to one dead-simple idea: **reward good behavior, punish bad behavior, repeat.** REINFORCE is just that idea, expressed precisely enough for a computer to execute.

1.10 Pages 11–12: A Problem with the Gradient and the Reward-to-Go Fix

Page 11 — Quiz: A Problem With the Gradient

This page gives a concrete worked example to expose a flaw hiding inside the gradient formula. The task: **teach a robot to walk forward.**

The reward is simple:

$$r(s, a) = \text{forward velocity of the robot (negative if it goes backwards)}$$

The agent runs 5 trajectories (τ^1 through τ^5):

Trajectory	What happened	Reward
τ^1	Falls backwards	Negative
τ^2	Falls forward	Small positive
τ^3	Stands still	Zero
τ^4	One small step forward, then falls backwards	Mixed
τ^5	One large step backwards, then small step forwards	Negative overall

The question the slide asks: given these 5 trajectories and the gradient formula from page 9, **what will the gradient actually encourage the policy to do?**

The answer: **it will encourage the robot to fall forward — not to actually take a step forward.**

Why? The flaw explained.

τ^2 (falls forward) gets a positive total reward because at least it was moving in the right direction for a moment. τ^4 (takes a step, then falls backward) might net out to a small or zero total reward because the backward fall cancels the forward step.

So when the gradient formula multiplies each trajectory's actions by its **total reward**, τ^2 gets a stronger positive push than τ^4 . The gradient is essentially saying: “falling forward is better than stepping forward and then falling backward” — which is wrong for the actual goal of learning to walk.

This is a **credit assignment problem**: the total reward of a whole trajectory is being used to judge every single action inside it, even actions that happened early and had nothing to do with the final outcome.

Page 12 — Improving the Gradient (Reward-to-Go)

The key insight.

Policy behavior at time t does not affect rewards at time $t' < t$.

In plain English: **your action right now cannot change what already happened in the past.** If you fall at step 5, that cannot hurt or help your reward from steps 1–4. Those are already done. So it is unfair — and noisy — to judge an action at time t by the rewards that came **before** it. We should only hold an action responsible for the rewards that come **after** it (including the current moment). These future rewards are called the **reward-to-go**.

Old gradient (the flawed one).

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_i \left(\sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(a_t^i | s_t^i) \right) \left(\sum_{t=1}^T r(s_t^i, a_t^i) \right)$$

The second sum is the **total reward of the entire trajectory** — it judges every action by the sum of all rewards, past and future.

New gradient (the fix).

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_i \sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(a_t^i | s_t^i) \left(\sum_{t'=t}^T r(s_{t'}^i, a_{t'}^i) \right)$$

Symbol	Meaning
$\sum_{t=1}^T$	Outer sum now goes over each individual timestep t , not just trajectories
$\nabla_{\theta} \log \pi_{\theta}(a_t^i s_t^i)$	Same as before — how to nudge θ for action a_t
$\sum_{t'=t}^T r(s_{t'}^i, a_{t'}^i)$	Reward-to-go: rewards from time t onward only

Instead of judging action a_t by the total reward of the whole episode, we only judge it by the rewards it could have actually influenced — everything from time t to the end. This is fairer and makes the gradient less noisy, because past rewards that had nothing to do with action a_t are no longer polluting its signal.

Back to the walking robot. With reward-to-go, τ^4 (step forward, then fall back) gets properly evaluated. The forward step at time $t = 1$ gets credited with the rewards from $t = 1$ onward — which include the positive forward velocity from that step. The backward fall at time $t = 2$ only gets blamed for rewards from $t = 2$ onward. The gradient now correctly separates the good action from the bad one within the same trajectory, instead of blurring them together under one total reward number.

1.11 Quiz: What the Gradient Does — The Walking Robot (Page 13)

The Setup

A humanoid robot is learning to walk. The reward $r(s, a)$ is the robot's forward speed. Move forward $\rightarrow$ positive reward. Move backward $\rightarrow$ negative reward.

Step 1 — Collect sample episodes

The robot tries 4 different episodes under the current policy $\pi_{\theta}(a_t | s_t)$:

Episode	What happened	Reward
τ^1	Falls forwards	Positive (moved forward)
τ^2	Slowly stumbles forwards	Positive
τ^3	Steadily walks forwards	Positive
τ^4	Runs forwards	Positive

All four got **positive reward** because all of them moved forward.

Step 2 — Compute the gradient

$$\nabla_{\theta} J(\theta) \approx \sum_i^N \sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(a_t^i | s_t^i) \left(\sum_{t'=t}^T r(s_{t'}^i, a_{t'}^i) \right)$$

- $\nabla_{\theta} J(\theta)$ — the gradient of the total reward with respect to the policy parameters; the “which direction to improve” signal
- $\sum_i^N$ — sum over all N sampled episodes

- $\sum_{t=1}^T$ — sum over every time step in an episode
- $\nabla_{\theta} \log \pi_{\theta}(a_t^i | s_t^i)$ — how much each action’s probability should shift
- $\sum_{t'=t}^T r(s_{t'}^i, a_{t'}^i)$ — total future reward collected from time step t onwards (the “return”)

For every action taken in every episode, multiply “how much this action’s probability should change” by “how much reward followed from this point.” Average this over all episodes to get the gradient — the direction to nudge the policy.

The Surprising Answer

Since **all four episodes got positive reward**, the gradient gives a positive push to **all of them** — including the falling episode (τ^1). The gradient cannot tell the difference between “fall forward” and “run forward.” It just sees “positive reward $\rightarrow$ do more of that.” So the policy gets nudged toward falling forward, which is clearly not the ideal walking behavior.

This is the core problem: **policy gradient is high-variance** — its gradient estimates are noisy and sensitive to the overall scale of rewards. If everything gets positive reward, everything gets reinforced, even bad behavior.

1.12 Introducing the Baseline (Page 14)

The Core Idea

Instead of reinforcing an action whenever the reward is positive, reinforce it only when the reward is **better than average**. This is done by subtracting a **baseline** b from the reward.

The Modified Gradient Formula

$$\nabla_{\theta} J(\theta) = E_{\tau \sim p_{\theta}} [\nabla_{\theta} \log p_{\theta}(\tau) \cdot (r(\tau) - b)]$$

- $\nabla_{\theta} J(\theta)$ — gradient of the objective (direction to improve policy)
- $E_{\tau \sim p_{\theta}}[\cdot]$ — average over all episodes sampled from the current policy
- $\nabla_{\theta} \log p_{\theta}(\tau)$ — how much the probability of this episode’s actions should shift
- $r(\tau)$ — total reward of episode τ
- b — the baseline (a reference number we subtract)
- $(r(\tau) - b)$ — how much better or worse this episode was compared to the baseline

Instead of asking “did this episode get good reward?”, we now ask “did this episode do **better than the baseline**?” If yes, increase the probability of those actions. If no, decrease it. The baseline acts like a reference point — a “passing grade.”

What is the baseline?

The simplest choice is the **average reward** across all sampled episodes:

$$b = \frac{1}{N} \sum_i r(\tau)$$

- N — total number of sampled episodes
- $r(\tau)$ — reward of each episode
- b — their simple average

This is just computing the average score across all your robot’s attempts.

Applying it to the walking example

Say the average reward across the 4 episodes is some value b . Now:

- “Running forward” (τ^4) scores well above $b \rightarrow (r - b) > 0 \rightarrow$ get a **positive push** (do more of this)
- “Falling forward” (τ^1) scores below $b \rightarrow (r - b) < 0 \rightarrow$ get a **negative push** (do less of this)

Now the gradient correctly discourages falling and encourages running, even though both had positive raw rewards.

Does subtracting the baseline break anything?

No — and this is proved at the bottom of the slide. Subtracting a constant baseline leaves the **expected gradient unchanged** (it is unbiased). The proof shows:

$$E[\nabla_{\theta} \log p_{\theta}(\tau) \cdot b] = b \nabla_{\theta} \int p_{\theta}(\tau) d\tau = b \nabla_{\theta} \cdot 1 = 0$$

The key step: $\int p_{\theta}(\tau) d\tau = 1$ because all probabilities must sum to 1. The gradient of a constant (1) is zero. So the baseline contributes **zero** to the expected gradient — it does not bias the direction of learning at all. But it **reduces variance** significantly, because individual episode rewards are now measured relative to the average instead of in absolute terms. Smaller swings in the gradient = more stable learning.

1.13 Quiz: Sparse Rewards — Jacket Folding (Page 15)

The Setup

A robot is learning to fold a jacket. The reward has only three possible values:

Outcome	Reward
Doesn’t touch the jacket	0
Folded with wrinkles	0.5
Neatly folded	1

Step 1 — Sample episodes

Episode	What happened	Reward
τ^1	Doesn't touch the jacket	0
τ^2	Folds only the sleeves	≈ 0
τ^3	Flattens but doesn't fold	0
τ^4	Folds the jacket	1

Three out of four episodes got zero reward. Only τ^4 succeeded.

Step 2 — Compute the gradient with baseline

$$\nabla_{\theta} J(\theta) \approx \sum_i^N \sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(a_t^i | s_t^i) \left(\sum_{t'=t}^T r(s_{t'}^i, a_{t'}^i) - b \right)$$

- $\nabla_{\theta} J(\theta)$ — the gradient; tells us which direction to nudge the policy
- $\nabla_{\theta} \log \pi_{\theta}(a_t^i | s_t^i)$ — how much each action's probability should shift
- $\sum_{t'=t}^T r(s_{t'}^i, a_{t'}^i)$ — total future reward from step t onward in episode i
- b — the baseline (average reward across all episodes, which is close to 0 here)

For each action taken in each episode, push that action's probability up or down based on how much better-than-average the outcome was. Subtract the baseline b so only above-average results get reinforced.

What happens in practice here?

Since most episodes scored 0 and the baseline $b \approx 0$, we get $(r - b) \approx 0$ for τ^1, τ^2, τ^3 — their gradient contributions are essentially **zero**. Only τ^4 , which scored 1, gives a meaningful signal.

So the gradient correctly points toward folding behavior — but the problem is it barely received any signal at all. Three out of four episodes contributed almost nothing to learning. This is called the **sparse reward problem**: when success is rare, most of your data is useless, the gradient is nearly flat most of the time, and learning is painfully slow and noisy. The baseline helped in the walking example (page 13) but did not fully fix this new problem.

1.14 Vanilla Policy Gradient and the On-Policy Problem (Page 16)

The Full Algorithm

1. **Sample** episodes $\{\tau^i\}$ from the current policy $\pi_{\theta}(a_t | s_t)$
2. **Compute** the gradient $\nabla_{\theta} J(\theta)$ using those episodes
3. **Update** the parameters
4. **Repeat** until the policy stops improving (convergence)

The update equation (Step 3):

$$\vec{\theta} = \vec{\theta} + \alpha \nabla_{\theta} J(\theta)$$

- $\vec{\theta}$ — the policy’s parameters (all the knobs controlling what actions the policy picks)
- α — the learning rate; how big a step to take (too big $\rightarrow$ overshoot, too small $\rightarrow$ crawl)
- $\nabla_{\theta} J(\theta)$ — the gradient; the direction that increases total reward

Take the current parameters $\vec{\theta}$, and nudge them in the direction of the gradient by a small amount α . Repeat. This is plain gradient ascent — climbing uphill on the reward surface one step at a time.

The Big Problem: On-Policy Learning

Vanilla policy gradient is **on-policy**, meaning the episodes used to compute the gradient in Step 2 must have been collected by the **exact current version of the policy**. Once you update θ in Step 3, those episodes are immediately outdated and thrown away. You can never reuse them.

This creates a painful cycle: collect data $\rightarrow$ compute gradient $\rightarrow$ update policy $\rightarrow$ **throw all data away** $\rightarrow$ collect new data $\rightarrow \dots$

Every single gradient step requires a fresh batch of episodes. If collecting data is slow (a real robot trying things in the physical world, or an expensive simulation), this wastes enormous amounts of time.

The natural question is: **what if we want to reuse data collected by an older version of the policy?** The answer is **off-policy RL using importance sampling** — a mathematical correction that lets you reuse old data by accounting for the fact that the old policy and the new policy chose actions differently. This is what the next slides address.

1.15 Off-Policy Policy Gradient (Page 17)

The Big Idea: Importance Sampling

Imagine you want to know what score a student would get on a different exam, but you only have their answers from the original exam. You can still estimate it — but you have to *adjust* each answer’s weight based on how different the two exams are. That adjustment factor is called the **importance sampling ratio**.

Same idea here: we collected episodes using the *old* policy θ , but we want to compute the gradient for a *new* (updated) policy θ' . We correct for this mismatch by multiplying by the ratio of the two policies’ probabilities.

Equation 1 — The Exact Off-Policy Gradient

$$\nabla_{\theta'} J(\theta') = \mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[\prod_{t=1}^T \frac{\pi_{\theta'}(a_t | s_t)}{\pi_{\theta}(a_t | s_t)} \left(\sum_{t=1}^T \nabla_{\theta'} \log \pi_{\theta'}(a_t | s_t) \right) \left(\sum_{t'=t}^T r(s_{t'}, a_{t'}) - b \right) \right]$$

- θ' — the **new** policy’s parameters (the one we are trying to improve)
- θ — the **old** policy’s parameters (the one that actually collected the data)

- $\mathbb{E}_{\tau \sim p_\theta(\tau)}[\cdot]$ — average over episodes collected using the **old** policy θ
- $\prod_{t=1}^T \frac{\pi_{\theta'}(a_t | s_t)}{\pi_\theta(a_t | s_t)}$ — the **importance sampling correction**: a product of ratios, one per time step; each ratio asks “how much more or less likely would the new policy have been to take this same action?”
- $\nabla_{\theta'} \log \pi_{\theta'}(a_t | s_t)$ — how much to shift each action’s probability under the new policy
- $\sum_{t'=t}^T r(s_{t'}, a_{t'}) - b$ — future reward from step t onward, minus the baseline

This is the REINFORCE gradient formula from before, but now with a correction factor bolted on — the big product in front. It uses data from the old policy, but multiplies each episode’s contribution by how differently the new policy would have behaved throughout that episode, step by step.

The Problem with Equation 1

The correction factor is a **product of T ratios** — one ratio multiplied together for every time step in the episode. If the episode is long (T is large), this product becomes unstable:

- If each ratio is slightly less than 1 $\rightarrow$ the product shrinks toward **zero** (gradient vanishes — no learning signal)
- If each ratio is slightly greater than 1 $\rightarrow$ the product explodes toward **infinity** (gradient blows up — wild, unstable updates)

This is like compounding interest but in a dangerous way. Multiplying together 100 numbers that are each 0.9 gives $0.9^{100} \approx 0.000027$ — basically nothing.

Equation 2 — Switching from Trajectories to Timesteps

$$\nabla_{\theta'} J(\theta') \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^T \frac{\pi_{\theta'}(s_{i,t}, a_{i,t})}{\pi_\theta(s_{i,t}, a_{i,t})} \nabla_{\theta'} \log \pi_{\theta'}(a_{i,t} | s_{i,t}) \left(\sum_{t'=t}^T r(s_{i,t'}, a_{i,t'}) - b \right)$$

- $\frac{\pi_{\theta'}(s_{i,t}, a_{i,t})}{\pi_\theta(s_{i,t}, a_{i,t})}$ — the ratio of how likely it was to **be in state s and take action a** under the new policy versus the old policy (a single ratio per timestep, not a product over all steps)
- $s_{i,t}, a_{i,t}$ — the state and action at time step t in episode i
- Everything else is the same as Equation 1

Instead of one giant product across the whole trajectory, we now use a **single ratio per time step** — comparing the new and old policy just at that one moment. This avoids the vanishing/exploding problem because there is no long chain of multiplications. The slide says this is “much less likely to explode/vanish.” However, the ratio $\frac{\pi_{\theta'}(s_{i,t}, a_{i,t})}{\pi_\theta(s_{i,t}, a_{i,t})}$ is the probability of both being in state s *and* taking action a . The state part is hard to compute — we do not know exactly how often the new policy would visit each state, because state visits depend on the full history of decisions. It is “hard to measure.”

Equation 3 — The Final Practical Form

$$\nabla_{\theta'} J(\theta') \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^T \frac{\pi_{\theta'}(a_{i,t} | s_{i,t})}{\pi_{\theta}(a_{i,t} | s_{i,t})} \nabla_{\theta'} \log \pi_{\theta'}(a_{i,t} | s_{i,t}) \left(\sum_{t'=t}^T r(s_{i,t'}, a_{i,t'}) - b \right)$$

- $\frac{\pi_{\theta'}(a_{i,t}|s_{i,t})}{\pi_{\theta}(a_{i,t}|s_{i,t})}$ — the ratio of **just the action probabilities**: given that we are already in state s , how much more or less likely is the new policy to pick action a compared to the old policy? The state part is dropped entirely.
- Everything else is the same as Equation 2

The state visitation ratio (the hard-to-measure part) is dropped, and only the action probability ratio is kept. This is an approximation — we are assuming the new and old policy visit states at roughly the same frequency — but it makes the formula practical. Both $\pi_{\theta'}(a | s)$ and $\pi_{\theta}(a | s)$ are just policy outputs that can be computed directly by plugging the state into the policy network.

Putting It All Together

The three equations represent a progression of trade-offs:

Equation	Correction factor	Explode/vanish?	Easy to compute?
Eq. 1	Product over all T steps	Yes, badly	Yes
Eq. 2	Single state-action ratio per step	No	No (state part is hard)
Eq. 3	Single action-only ratio per step	No	Yes

Equation 3 is the final usable formula. It lets you reuse data from an old policy, corrects for the mismatch using a simple per-step ratio, and is fully computable. This is the foundation for **off-policy** algorithms that are far more data-efficient than vanilla policy gradient.

1.16 Off-Policy Policy Gradient: Full Algorithm (Page 18)

The Algorithm (same three steps, different loop)

1. Sample episodes $\{\tau^i\}$ from policy $\pi_{\theta}(a_t | s_t)$
2. Compute $\nabla_{\theta} J(\theta)$ using those episodes
3. Update: $\vec{\theta} = \vec{\theta} + \alpha \nabla_{\theta} J(\theta)$

The update equation:

$$\vec{\theta} = \vec{\theta} + \alpha \nabla_{\theta} J(\theta)$$

- $\vec{\theta}$ — the policy's parameters (all the internal settings that control behavior)
- α — the learning rate; how big each step is
- $\nabla_{\theta} J(\theta)$ — the gradient; the direction that increases reward

Nudge the parameters in the direction that increases reward, scaled by the step size α . This is the same update as vanilla policy gradient.

What is actually different: the loop

The slide shows two arrows on the algorithm:

- **Red arrow** (on-policy loop): After step 3, loop back all the way to step 1 — collect **new** data before doing anything else.
- **Orange arrow** (off-policy loop): After step 3, loop back only to step 2 — recompute the gradient on the **same old data** using the importance sampling correction from page 17, and update again.

In vanilla policy gradient, after one update you must throw away all your data and go collect more. In off-policy policy gradient, you can reuse the same batch and do **multiple gradient steps** on it — because the importance sampling ratio in the gradient formula already corrects for the fact that the policy has changed since the data was collected.

Think of it like studying for an exam: on-policy is like solving one practice problem and then immediately going to get a new set — very inefficient. Off-policy is like solving the same set of practice problems multiple times, learning more from them each pass through.

1.17 The KL Divergence Constraint (Page 19)

The Problem

When you take many gradient steps on the same batch of old data, the new policy θ' can drift very far from the old policy θ that collected the data. But the importance sampling correction only works well when the two policies are *similar*. Once they diverge significantly:

- The old data no longer reflects the states the new policy would actually visit
- The gradient estimate becomes increasingly wrong
- You might make a bad update that seriously hurts the policy's performance — and you might not even realize it until it is too late

It is like navigating using an old map. A slightly outdated map is fine. But if the roads have completely changed, the map will lead you off a cliff.

The Fix — Constrain How Much the Policy Can Change

$$\mathbb{E}_{s \sim \pi_\theta} [D_{\text{KL}}(\pi_{\theta'}(\cdot | s) \parallel \pi_\theta(\cdot | s))] \leq \delta$$

- $\mathbb{E}_{s \sim \pi_\theta} [\cdot]$ — average over states that the **old** policy θ visited; we are measuring the difference in the situations the old policy actually experienced
- $D_{\text{KL}}(\cdot \parallel \cdot)$ — **KL divergence**: a mathematical ruler for measuring how different two probability distributions are; it equals 0 when they are identical, and grows larger as they diverge
- $\pi_{\theta'}(\cdot | s)$ — the **new** policy's action choices at state s (a full probability distribution over actions)
- $\pi_\theta(\cdot | s)$ — the **old** policy's action choices at state s
- δ — a small threshold (your “budget” for how much the policy is allowed to change); typically a small number like 0.01

On average across all the states the old policy visited, the new policy must not be “too different” from the old one. “Too different” is measured by KL divergence, and the constraint says it must stay at or below the budget δ . This limits how large each policy update can be, keeping the new policy close enough to the old one that the recycled data remains trustworthy.

Why This Matters

This constraint is the core idea behind **Trust Region Policy Optimization (TRPO)** — one of the most important algorithms in modern RL. The name comes from this: you trust the old data only within a certain region of policy space. Inside that region (where new and old policies are close), the gradient estimate is reliable. Outside it, you cannot trust the old data anymore and must stop.

A simpler version of this same idea later became **PPO (Proximal Policy Optimization)**, which enforces a similar limit without the full math of a trust region, and is today one of the most widely used RL algorithms in practice — including in training large language models like the ones behind ChatGPT.

2 Actor Critic Method

What is value function and what is Q function.

Value Function $V^\pi(s)$

What it answers: “How good is it to *be* in state s ?”

More precisely: if you’re currently in state s and you follow policy π from that point forward, what is the **expected total (sum of future) reward** you’ll collect?

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t r_t \mid s_0 = s \right]$$

It doesn’t care which action you take — it averages over whatever actions π would naturally choose. So $V^\pi(s)$ is purely a **state-level** signal.

Q-Function $Q^\pi(s, a)$

What it answers: “How good is it to be in state s , take a *specific* action a , and then follow π ?”

The key difference: you **commit to a specific action first**, then hand control back to the policy.

$$Q^\pi(s, a) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t r_t \mid s_0 = s, a_0 = a \right]$$

This is a **state-action level** signal — it lets you compare individual actions against each other.

The Key Relationship

$$V^\pi(s) = \mathbb{E}_{a \sim \pi(\cdot|s)} [Q^\pi(s, a)]$$

$V^\pi(s)$ is just $Q^\pi(s, a)$ **averaged over all actions weighted by the policy**. V sees the average picture; Q sees the picture for each individual action.

Why Both Exist

This distinction is what makes the **Advantage function** possible:

$$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s)$$

A^π tells you how much better (or worse) action a is compared to what the policy would do on average at state s . This is the core signal used by the **Critic** in Actor-Critic methods to give the Actor more informative feedback than raw rewards.

How V is Q averaged over all actions? Explain this with a concrete example.

Concrete Example

Say you're at state s , and your policy π has **3 possible actions** with these probabilities and Q -values:

Action	$\pi(a s)$ — probability	$Q^\pi(s, a)$ — expected future reward
a_1	0.5	10
a_2	0.3	4
a_3	0.2	1

What $V^\pi(s)$ Is

$V^\pi(s)$ asks: “If I just *run policy* π from state s — without picking any specific action myself — what reward do I expect?”

Since π will pick a_1 50% of the time, a_2 30%, a_3 20%, the expected reward is just the **probability-weighted sum of Q -values**:

$$V^\pi(s) = 0.5 \times 10 + 0.3 \times 4 + 0.2 \times 1 = 5 + 1.2 + 0.2 = \mathbf{6.4}$$

That's it. V doesn't have any secret information — it's literally just averaging $Q^\pi(s, a)$ weighted by how often the policy *would* choose each action.

Why This Makes Intuitive Sense

- $Q^\pi(s, a_1) = 10 \rightarrow$ if you *commit* to a_1 , you expect 10.
- But you don't always take a_1 . Half the time you do, and the other half you take worse actions.
- So the **actual expected reward from state s** — without fixing an action — is 6.4, which is lower than the best possible Q .

$V^\pi(s) = 6.4$ is the **average performance of the policy at state s** . $Q^\pi(s, a)$ is the performance of **one specific choice**.

The Advantage Connection

This is why the Advantage function is meaningful:

- $A^\pi(s, a_1) = 10 - 6.4 = +3.6 \rightarrow a_1$ is better than average
- $A^\pi(s, a_2) = 4 - 6.4 = -2.4 \rightarrow a_2$ is worse than average
- $A^\pi(s, a_3) = 1 - 6.4 = -5.4 \rightarrow a_3$ is much worse than average

V acts as the **baseline**. Q minus that baseline tells you which actions are actually worth doing more often — which is exactly the signal the Actor uses to improve.

What does A tell us?

$A^\pi(s, a)$ tells you: **“Is this action better or worse than what the policy would do on average at this state — and by how much?”**

- $A > 0 \rightarrow$ this action is better than average. Do it more.
- $A < 0 \rightarrow$ this action is worse than average. Do it less.
- $A = 0 \rightarrow$ this action is exactly as good as following the policy blindly.

From the previous example:

- a_1 has $A = +3.6 \rightarrow$ push the policy to favor a_1 more
- a_2 has $A = -2.4 \rightarrow$ push the policy away from a_2
- a_3 has $A = -5.4 \rightarrow$ strongly push away from a_3

The key insight: **raw reward tells you if something was good in absolute terms. Advantage tells you if it was good *relative to your other options***. That relative comparison is what makes the gradient update meaningful — instead of just chasing high-reward actions, the agent learns which actions are genuinely better than its current behavior.

Issues with Policy Gradient Methods

The Big Idea

This slide is asking: **“What’s wrong with how the AI currently learns?”** The answer is — it’s clumsy. It blames or rewards the *entire sequence of actions* together, even when only one action was actually good or bad. This causes two big problems, shown through two examples.

The Equation

$$\nabla_\theta J(\theta) \approx \sum_i \sum_{t=1}^T \nabla_\theta \log \pi_\theta(a_t^i | s_t^i) \left(\sum_{t'=t}^T r(s_{t'}, a_{t'}^i) - b \right)$$

What each symbol means:

Symbol	What it is
$\nabla_{\theta} J(\theta)$	The direction to tweak the policy to get better rewards
θ	The knobs inside the AI's brain (its neural network weights)
$\sum_i$	"Add this up across all tries (trajectories)"
$\sum_t$	"Add this up across every time step in one try"
$\log \pi_{\theta}(a_t^i s_t^i)$	How likely the policy thought action a was in state s
$\sum_{t'=t} r(\dots)$	Total reward collected from this moment forward (reward to go)
b	A baseline — the average reward, subtracted to reduce noise

What the equation is saying, in plain English:

For every action the AI took, look at all the rewards it collected *after* that action. If those future rewards were high, nudge the AI to make that action more likely next time. If they were low, nudge it to avoid that action. Subtract b so you're comparing to the average, not reacting to raw numbers.

Problem 1: The Walking Robot

The robot takes **one small step forward**, then **falls backwards**. The total reward for the trip is negative (it fell!).

The equation sees the negative total reward and says: "*That step forward must have been bad — do it less!*"

But that's **wrong**. The step forward was actually a good move. The fall was the problem. The algorithm can't tell the difference — it punishes the whole sequence including the good part.

- × It pushes down the chance of "step forward" even though stepping forward was fine.
- × It wastes the data because it learned the wrong lesson.

Problem 2: The Jacket-Folding Robot

Three different practice runs (trajectories):

- τ^2 — folds only the sleeves
- τ^3 — flattens the jacket only
- τ^4 — fully folds the jacket (gets reward)

Only τ^4 gets a reward of 1. τ^2 and τ^3 get **zero reward** because the task wasn't fully done.

The algorithm sees zero reward for τ^2 and τ^3 and says: "*Nothing useful happened there, ignore it.*"

But that's **also wrong**. Folding the sleeves and flattening the jacket are *progress* — they were good intermediate steps that the AI should learn to repeat.

- × With sparse rewards, the AI wastes most of its practice runs.
- ✓ The question the slide ends on: **Can we teach the AI to know what's good and bad, even mid-task?** — That's exactly what Actor-Critic methods solve.

Improving Policy Gradients (Page 5)

The starting equation (from last time):

$$\nabla_{\theta} J(\theta) \approx \sum_i \sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(a_t^i | s_t^i) \left(\sum_{t'=t}^T r(s_{t'}^i, a_{t'}^i) - b \right)$$

Symbol	What it means
$\nabla_{\theta} J(\theta)$	The direction to improve the policy — like a compass saying “adjust your brain th
θ	The AI’s brain settings (numbers inside the neural network)
$\sum_i$	Add this up across all practice runs
$\sum_{t=1}^T$	Add this up across every step of one run
$\nabla_{\theta} \log \pi_{\theta}(a_t^i s_t^i)$	How confident the AI was in the action it chose at that step
$\sum_{t'=t}^T r(\dots)$	All the rewards collected <i>from this step forward</i> (called reward to go)
b	A baseline — the average reward — subtracted to reduce noise

The “reward to go” part is trying to answer: “*If I took this action in this situation, how much total reward did I collect afterward?*” That quantity — the expected future reward of taking action a from state s — is exactly the definition of the **Q-function**. So the slide says:

Step 1 — Reward to go equals the Q-function:

$$\sum_{t=t'}^T \mathbb{E}_{\pi_{\theta}}[r(s_{t'}, a_{t'} | s_t, a_t)] = Q_{\pi}(s_t, a_t)$$

Symbol	What it means
$\mathbb{E}_{\pi_{\theta}}[\cdot]$	The average outcome if you repeat this many times under policy π
$r(s_{t'}, a_{t'})$	Reward you get at each future step
$Q_{\pi}(s_t, a_t)$	The Q-function: expected total future reward for taking action a_t in state s_t

Instead of using the messy, noisy actual rewards from one run, we can use the Q-function — which is the *average* expected reward. The diagram on the slide shows this visually: from state s , you take action a , then let policy π run freely. All those possible future paths spread out like branches, and Q is the expected sum of rewards across all of them. One single run (the “reward to go”) is just one noisy sample of Q.

Step 2 — The baseline b turns out to be the Value function:

$$b = \text{average reward} = \frac{1}{N} \sum_i Q_{\pi}(s_{i,t}, a_{i,t}) = V_{\pi}(s_t)$$

Symbol	What it means
$\frac{1}{N} \sum_i$	Average across all N practice runs
$Q_{\pi}(s_{i,t}, a_{i,t})$	Q-value for each run’s action at this step
$V_{\pi}(s_t)$	The Value function — the average Q across all possible actions the policy would take

The baseline b was supposed to be “the average reward” to compare against. When you average Q over all the actions the policy would naturally take, you get exactly $V_\pi(s_t)$ — the Value function. So b isn’t arbitrary; it’s the average quality of the current state.

Step 3 — The whole thing simplifies to the Advantage:

$$\nabla_\theta J(\theta) \approx \sum_i \sum_{t=1}^T \nabla_\theta \log \pi_\theta(a_t^i | s_t^i) A_\pi(s_{i,t}, a_{i,t})$$

Symbol	What it means
$A_\pi(s_{i,t}, a_{i,t})$	The Advantage — how much better this specific action was compared to the average

Since $Q - b = Q - V = A$ (the Advantage function), the whole messy reward-to-go-minus-baseline expression collapses into just A_π . The slide is saying: instead of asking “did I get good rewards?”, ask “was this action *better than average* for this situation?” — that’s a much smarter, cleaner signal for improving the policy. Actions better than average get a positive score $\rightarrow$ do them more. Actions worse than average get a negative score $\rightarrow$ do them less.

How using A is different from using `reward_to_go - b`? `reward_to_go = Q` and `b = V`. I see no difference.

You’re right — mathematically they are **identical**. $A_\pi = Q - V = \text{reward_to_go} - b$, by definition. No difference on paper.

The difference is in **how you compute them in practice**.

`reward_to_go - b` (pure policy gradient way):

You run a full trajectory, collect actual rewards, sum them up. That sum *is* your estimate of Q . You subtract the average across runs as b . Both numbers come from **raw sampled rewards** — noisy, because any single run is just one random path through the environment.

A via the Critic (actor-critic way):

You train a **neural network** to learn $V(s)$. Then you compute the advantage as:

$$\hat{A}_t = r_t + V_\phi(s_{t+1}) - V_\phi(s_t)$$

You’re no longer summing up a long noisy chain of rewards. The network gives you a smooth, learned estimate of value at any state.

The practical consequence:

	<code>reward_to_go - b</code>	Critic-estimated A
Source	Raw rewards from full runs	Neural network estimates
Variance	High	Lower
Bias	Unbiased	Can be biased if network is wrong
Data needed	Many full trajectories	Can learn from shorter, incomplete ones

So the slide isn't introducing a new *math* — it's showing that A is the right **target concept**, and the whole point of actor-critic is to **estimate A better** than raw reward sums allow. The Critic exists precisely to give you a lower-variance estimate of A than you'd get by just running trajectories.

Online Reinforcement Learning (Page 6)

This slide shows the **big picture loop** of how the AI actually trains. It has three stages that repeat forever.

Step 0 — Start somewhere:

The policy π is initialized — either randomly (the AI does dumb stuff at first), by copying a human expert (imitation learning), or using some hand-crafted rules (heuristics).

Then the loop runs:

Stage 1 → Run policy to collect data.

Let the AI play in the environment for a while. Record what happened: which states it visited, which actions it took, what rewards it got. This is the raw experience batch.

Stage 2 → Fit a model to estimate expected return.

Use that collected data to train a neural network to estimate V^π , Q^π , or A^π — i.e., teach the Critic how good each state or action was. This is the “how good was that?” step.

Stage 3 → Improve the policy:

$$\theta \leftarrow \theta + \nabla_\theta J(\theta)$$

Symbol	What it means
θ	The AI's brain settings (neural network weights)
$\leftarrow$	“Update this to become”
$\nabla_\theta J(\theta)$	The direction that makes the policy better

Take a step in the direction that improves the policy, using the Advantage estimates from Stage 2. Then go back to Stage 1 with the new, slightly better policy. Repeat.

Estimating Expected Return (Page 7)

This slide answers: how do we actually compute A_π ? We start from its definition and simplify it into something we can calculate.

Step 1 — Definition of Advantage:

$$A_\pi(s_t, a_t) = Q_\pi(s_t, a_t) - V_\pi(s_t)$$

Symbol	What it means
$A_\pi(s_t, a_t)$	How much better action a_t is compared to the average action at state s_t
$Q_\pi(s_t, a_t)$	Expected total future reward if you take action a_t at state s_t , then follow π
$V_\pi(s_t)$	Expected total future reward at state s_t if you just follow π normally

Advantage is simply Q minus V — how much better (or worse) your specific action was compared to what the policy would do on average.

Step 2 — Expand Q:

$$Q_\pi(s_t, a_t) = \sum_{t'=t}^T \mathbb{E}_{\pi_\theta}[r(s_{t'}, a_{t'} \mid s_t, a_t)]$$

This is just the definition of Q — the expected sum of all future rewards from this step onward. Nothing new here, just spelling it out.

Step 3 — Rewrite Q using just one step + V:

$$Q_\pi(s_t, a_t) = r(s_t, a_t) + \mathbb{E}_{s_{t+1} \sim p(\cdot | s_t, a_t)}[V_\pi(s_{t+1})]$$

Symbol	What it means
$r(s_t, a_t)$	The immediate reward you get right now
$\mathbb{E}_{s_{t+1} \sim p(\cdot s_t, a_t)}[\cdot]$	The average over all possible next states you could land in
$V_\pi(s_{t+1})$	The value of wherever you end up next

Instead of summing rewards all the way to the end of the episode, we can split it: the reward you get *right now*, plus the value of the *next state* (which already accounts for everything after). This is a much shorter calculation.

Step 4 — Approximate by using the actual next state:

$$Q_\pi(s_t, a_t) \approx r(s_t, a_t) + V_\pi(s_{t+1})$$

We drop the expectation and just use the real s_{t+1} that actually happened in the run. This is an approximation — one sample instead of averaging over all possible next states — but it works well enough in practice.

Step 5 — Final practical formula for Advantage:

$$A_\pi(s_t, a_t) \approx r(s_t, a_t) + V_\pi(s_{t+1}) - V_\pi(s_t)$$

Symbol	What it means
$r(s_t, a_t)$	Reward you got right now
$V_\pi(s_{t+1})$	How good the next state is (estimated by the Critic network)
$V_\pi(s_t)$	How good the current state was expected to be (estimated by the Critic network)

This is the key result. To compute A , you only need: the reward you just got, plus what the Critic says about the next state, minus what the Critic says about the current state. No long reward chains. No waiting until the end of the episode. This difference $r + V(s_{t+1}) - V(s_t)$ is called the **TD error** (Temporal Difference error). The slide closes by asking: but how do we estimate $V_\pi(s_t)$? — that's answered on the next slides using Monte Carlo or TD learning.

Actor-Critic Algorithm (Page 8)

This slide puts it all together as a concrete step-by-step algorithm. Two networks are running: the **Actor** (the policy, picks actions) and the **Critic** (the value function, judges states).

Step 1 — Sample a batch of data:

$$\{(s_{1,i}, a_{1,i}, \dots, s_{T,i}, a_{T,i})\} \text{ from } \pi_\theta$$

Run the current policy in the environment and record everything that happened across N trajectories: every state visited, every action taken, every reward received.

Step 2 — Fit the Critic:

Train the value network $\hat{V}_\phi^{\pi_\theta}$ on the collected data — i.e., teach it to predict the total future reward from any state it saw. ϕ are the Critic's own internal weights, separate from the Actor's θ .

Step 3 — Compute the Advantage estimate:

$$\hat{A}^{\pi_\theta}(s_{i,t}, a_{i,t}) = r(s_{i,t}, a_{i,t}) + \hat{V}_\phi^{\pi_\theta}(s_{i,t+1}) - \hat{V}_\phi^{\pi_\theta}(s_{i,t})$$

Symbol	What it means
$\hat{A}^{\pi_\theta}$	Estimated advantage — the hat $\hat$ means it's an approximation, not exact
$r(s_{i,t}, a_{i,t})$	Actual reward received at this step
$\hat{V}_\phi^{\pi_\theta}(s_{i,t+1})$	Critic's estimate of how good the <i>next</i> state is
$\hat{V}_\phi^{\pi_\theta}(s_{i,t})$	Critic's estimate of how good the <i>current</i> state is

This is the TD error from page 7 — immediate reward plus next-state value minus current-state value. Positive means this action was better than expected; negative means it was worse.

Step 4 — Compute the policy gradient:

$$\nabla_\theta J(\theta) \approx \sum_i^N \sum_{t=1}^T \nabla_\theta \log \pi_\theta(a_t^i | s_t^i) \hat{A}^{\pi_\theta}(s_{i,t}, a_{i,t})$$

Same formula as before — but now the Advantage is estimated by the Critic network, not from raw noisy reward sums.

Step 5 — Update the Actor:

$$\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$$

Symbol	What it means
α	Learning rate — how big a step to take (a small number like 0.001)
$\nabla_{\theta} J(\theta)$	The direction that improves the policy

Nudge the Actor’s weights in the direction that makes good-advantage actions more likely. Then go back to Step 1 with the updated policy.

The diagram on the right shows the data flow: the environment sends a *state* to both the Actor and Critic. The Actor outputs an *action*. The environment responds with a *reward* and a new state. The Critic uses the reward and both states to compute the **TD error**, which is used to update itself (Critic training) and to provide the Advantage signal back to the Actor (Actor training).

Off-Policy Actor-Critic: Version 1 (Multiple Gradient Steps) & Too Many Gradient Steps

The Big Idea

Imagine a robot learning to play a video game. It tries some moves, records what happened, and then uses that recording to get smarter. But here’s the trick — it tries to learn *a lot* from that one recording before throwing it away. That’s what this page is about.

The algorithm has 5 steps:

Step 1 — Collect experiences. The robot (called the “actor,” running policy π_{θ}) plays through the game several times and records all the situations (states) and moves (actions) it made.

Step 2 — Train the Critic. A second brain called the “critic” ($\hat{V}_{\phi}$) looks at all those recordings and learns to estimate: “*From this situation, how much total reward can we expect?*”

Step 3 — Calculate the Advantage.

$$\hat{A}^{\pi_{\theta}}(s_{i,t}, a_{i,t}) = r(s_{i,t}, a_{i,t}) + \hat{V}_{\phi}^{\pi_{\theta}}(s_{i,t+1}) - \hat{V}_{\phi}^{\pi_{\theta}}(s_{i,t})$$

What each symbol means:

- $\hat{A}^{\pi_{\theta}}$ — The “advantage.” How much *better* (or worse) was this action compared to what we expected?
- $s_{i,t}$ — The situation (state) the robot was in, at time step t , in episode i .
- $a_{i,t}$ — The action the robot took in that situation.

- $r(s_{i,t}, a_{i,t})$ — The immediate reward earned right after taking that action.
- $\hat{V}_\phi^{\pi_\theta}(s_{i,t+1})$ — The critic’s estimate of future rewards from the *next* situation.
- $\hat{V}_\phi^{\pi_\theta}(s_{i,t})$ — The critic’s estimate of future rewards from the *current* situation.

What it’s saying: The advantage is like asking: “*Was this move smarter than average?*” You take the reward you actually got, plus how good the future looks from where you landed, then subtract how good the future was already expected to look from where you started. If that number is positive, the move was better than expected — a smart move. If negative, it was worse. Think of it like getting a surprise quiz score — the advantage is how much you beat (or missed) your expected grade.

Step 4 — Calculate how to improve the policy.

$$\nabla_{\theta'} J(\theta') \approx \sum_i \sum_{t=1}^T \frac{\pi_{\theta'}(a_{i,t}|s_{i,t})}{\pi_\theta(a_{i,t}|s_{i,t})} \nabla_{\theta'} \log \pi_{\theta'}(a_t^i | s_t^i) \hat{A}^{\pi_\theta}(s_{i,t}, a_{i,t})$$

What each symbol means:

- $\nabla_{\theta'} J(\theta')$ — The direction to nudge the new policy θ' to make it better (the gradient).
- $\pi_{\theta'}(a_{i,t}|s_{i,t})$ — The probability the *new* policy would pick that same action in that situation.
- $\pi_\theta(a_{i,t}|s_{i,t})$ — The probability the *old* policy picked that action (the one that collected the data).
- The fraction — The *importance weight*. It’s a correction factor.
- $\nabla_{\theta'} \log \pi_{\theta'}(\cdot)$ — How sensitive the new policy is to small changes, for that particular action.
- $\hat{A}^{\pi_\theta}(\cdot)$ — The advantage we computed in Step 3.
- $\sum_i \sum_{t=1}^T$ — Sum this up across all episodes (i) and all time steps ($t = 1$ to T).

What it’s saying: We collected data using the *old* policy, but we want to improve the *new* policy. The problem is the data was collected under different rules than the ones we’re now updating. So we use the fraction (importance weight) as a *correction factor* — like converting currencies. If the new policy would have chosen that action much more often than the old policy did, the weight is high, meaning that action gets more credit. The whole equation then pushes the new policy to do more of the things that had a high advantage, adjusted fairly for the difference between old and new behavior.

Step 5 — Update the policy. Move the policy in the direction that makes it better, scaled by a learning rate α :

$$\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$$

The Warning: If you do Steps 4–5 too many times on the same old data, things break.

The Problem — Too Many Gradient Steps

Imagine you studied for a test using last week’s notes, but the test is actually on new material. The more you study those old notes, the more confident you get — but in the *wrong* direction. The same thing happens here: if you update the policy too many times using old data, the new policy drifts so far from the old one that the importance weights become useless lies. The new policy “overfits” to the old data.

Fix 1 — Put a Leash on the Policy (KL Constraint)

$$\mathbb{E}_{s \sim \pi_\theta} [D_{\text{KL}}(\pi_{\theta'}(\cdot | s) \parallel \pi_\theta(\cdot | s))] \leq \delta$$

What each symbol means:

- $\mathbb{E}_{s \sim \pi_\theta}$ — The average, taken over all situations s that the old policy π_θ would visit.
- $D_{\text{KL}}(\cdot \parallel \cdot)$ — KL divergence: a way to measure *how different* two probability distributions are. Think of it as the “distance” between two sets of rules.
- $\pi_{\theta'}(\cdot | s)$ — The new policy’s behavior in situation s .
- $\pi_\theta(\cdot | s)$ — The old policy’s behavior in situation s .
- $\leq \delta$ — The difference must stay below a small limit δ (delta), a number you set in advance.

What it’s saying: This equation is a rule that says: “*The new policy is not allowed to become too different from the old policy.*” You measure how different they are using KL divergence — a mathematical way of comparing two sets of decisions. If the new policy starts changing too drastically, you stop it. It’s like telling a student: “*You can revise your answers, but don’t change more than 10% of them.*” This idea is used in training modern AI language models (like ChatGPT) to make sure they don’t go off the rails during fine-tuning.

Fix 2 — Clip the Importance Weights (The PPO Idea)

Instead of directly measuring how different the policies are, what if we just *cap* the importance weight ($\pi_{\theta'}/\pi_\theta$) so it can never get too large or too small? This doesn’t stop the policy from changing, but it removes the *reward* for changing too much — the policy has no reason to drift far because it can’t exploit extreme importance weights anymore. This is the core idea behind **PPO (Proximal Policy Optimization)**, one of the most popular and widely used training algorithms in modern AI, including the training of large language models.

Surrogate Objective

The Big Idea

This page answers one smart question: “*Instead of computing a complicated gradient every time, can we find a simpler score to just maximize directly?*” The answer is yes — and that simpler score is called the **surrogate objective**.

Step 1 — Start with the gradient from page 9

$$\nabla_{\theta'} J(\theta') \approx \sum_i \sum_{t=1}^T \frac{\pi_{\theta'}(a_{i,t} | s_{i,t})}{\pi_{\theta}(a_{i,t} | s_{i,t})} \nabla_{\theta'} \log \pi_{\theta'}(a_t^i | s_t^i) \hat{A}^{\pi_{\theta}}(s_{i,t}, a_{i,t})$$

This was explained on page 9. The two key pieces being looked at more carefully here are: the importance weight fraction, and the log-gradient of the policy.

Step 2 — A Useful Math Trick

$$p_{\theta}(\tau) \nabla_{\theta} \log p_{\theta}(\tau) = \nabla_{\theta} p_{\theta}(\tau)$$

What each symbol means:

- $p_{\theta}(\tau)$ — The probability of a sequence of events τ (like a full episode of play) under policy with parameters θ .
- $\nabla_{\theta} \log p_{\theta}(\tau)$ — How sensitive the *log* of that probability is to small changes in θ .
- $\nabla_{\theta} p_{\theta}(\tau)$ — How sensitive the *actual* probability is to small changes in θ .

What it's saying: This is a well-known math shortcut called the **log-derivative trick**. It says: multiplying a probability by the gradient of its own log is the same as just taking the gradient of the probability directly. Think of it like this: if you know the gradient of the “log score” and you also know the score itself, you can recover the gradient of the score without doing extra work. This lets us collapse the importance weight and the log-gradient into a single, cleaner expression — the gradient of the probability ratio. It's a pure algebra trick that rewrites one messy form into another simpler form.

Step 3 — The Surrogate Objective

Applying that trick to the gradient equation allows us to “un-wrap” the log and arrive at a plain function — not a gradient, but a score you can just maximize directly:

$$\tilde{J}(\theta') \approx \sum_{t,i} \frac{\pi_{\theta'}(a_{i,t} | s_{i,t})}{\pi_{\theta}(a_{i,t} | s_{i,t})} \hat{A}^{\pi_{\theta}}(s_{t,i}, a_{t,i})$$

What each symbol means:

- $\tilde{J}(\theta')$ — The surrogate objective. A fake (but useful) score that stands in for the real objective $J(\theta')$. The tilde signals it's a substitute.
- $\pi_{\theta'}(a_{i,t} | s_{i,t})$ — The probability that the *new* policy picks action a in situation s .
- $\pi_{\theta}(a_{i,t} | s_{i,t})$ — The probability that the *old* policy picked that same action.
- The fraction — The importance weight, correcting for the fact that data came from the old policy.
- $\hat{A}^{\pi_{\theta}}(s_{t,i}, a_{t,i})$ — The advantage: how much better that action was compared to average.

- $\sum_{t,i}$ — Sum over all time steps t and all episodes i .

What it’s saying: The surrogate objective $\tilde{J}(\theta')$ is a simplified, easy-to-maximize score. Instead of computing a complex gradient formula at every step, you just directly maximize this expression. It asks: *“For every action taken in the old recordings, how much more (or less) likely would the new policy be to make that same move, and was that move a good one?”* If the new policy gives higher probability to good moves (high advantage) and lower probability to bad ones (negative advantage), the score goes up. The key insight is that when the new policy is still close to the old one ($\theta' \approx \theta$), taking the gradient of $\tilde{J}$ gives you the exact same result as taking the gradient of the real objective J . So you can optimize $\tilde{J}$ as a stand-in — it’s mathematically equivalent at the point that matters, and much easier to work with.

The Two Warnings at the Bottom

- **Good news:** Maximizing $\tilde{J}$ naturally pushes the policy to take good actions more often — exactly what we want.
- **Bad news:** Because $\tilde{J}$ rewards giving higher probability to good actions, the policy is tempted to change *a lot* — drifting far from the old policy. If you run too many gradient steps on $\tilde{J}$ with the same old data, the ratio $\pi_{\theta'}/\pi_{\theta}$ grows huge and meaningless, and the policy overfits. This is precisely the “too many gradient steps” problem from page 10, and why PPO and KL constraints exist.

How did the log part of the initial equation vanish?

The log vanishes because of the math trick shown in the middle of the page. Here’s exactly how, step by step.

Look at the two boxed parts in Equation 1 together:

$$\frac{\pi_{\theta'}(a|s)}{\pi_{\theta}(a|s)} \cdot \nabla_{\theta'} \log \pi_{\theta'}(a|s)$$

The math trick says:

$$p(\tau) \cdot \nabla \log p(\tau) = \nabla p(\tau)$$

Apply that to the numerator only — $\pi_{\theta'}(a|s)$ is sitting in the fraction AND inside the log. So:

$$\pi_{\theta'}(a|s) \cdot \nabla_{\theta'} \log \pi_{\theta'}(a|s) = \nabla_{\theta'} \pi_{\theta'}(a|s)$$

Plug that back in:

$$\frac{\pi_{\theta'}(a|s) \cdot \nabla_{\theta'} \log \pi_{\theta'}(a|s)}{\pi_{\theta}(a|s)} = \frac{\nabla_{\theta'} \pi_{\theta'}(a|s)}{\pi_{\theta}(a|s)}$$

Since π_{θ} (the old policy) **doesn’t depend on** θ' , it’s just a constant with respect to the gradient. So you can pull $\nabla_{\theta'}$ outside:

$$= \nabla_{\theta'} \left[\frac{\pi_{\theta'}(a|s)}{\pi_{\theta}(a|s)} \right]$$

Now the full equation becomes:

$$\nabla_{\theta'} J(\theta') = \nabla_{\theta'} \sum_{t,i} \frac{\pi_{\theta'}(a_{t,i} | s_{t,i})}{\pi_{\theta}(a_{t,i} | s_{t,i})} \hat{A}^{\pi_{\theta}}(s_{t,i}, a_{t,i})$$

The right-hand side is just $\nabla_{\theta'}$ acting on $\tilde{J}(\theta')$. So:

$$\nabla_{\theta'} J(\theta') = \nabla_{\theta'} \tilde{J}(\theta')$$

The log didn't “vanish” — it got absorbed. The log-gradient and the importance weight were multiplied together, and the trick collapsed that product into a single clean gradient of the ratio. What's left inside the gradient sign is exactly $\tilde{J}$. Since the gradients match, you can just maximize $\tilde{J}$ directly instead of dealing with the log form.

Pages 12, 13, 14 — Proximal Policy Optimization (PPO)

All three pages are about one algorithm: PPO. Think of PPO as a smarter, safer way to train an AI agent.

Page 12 — PPO's First Two Tricks

Before the tricks, here is the starting point (the surrogate objective):

$$\tilde{J}(\theta') \approx \sum_{t,i} \frac{\pi_{\theta'}(a_{t,i} | s_{t,i})}{\pi_{\theta}(a_{t,i} | s_{t,i})} \hat{A}^{\pi_{\theta}}(s_{t,i}, a_{t,i})$$

What each symbol means:

- $\tilde{J}(\theta')$ — the score we are trying to maximize for the new policy
- θ' — the parameters (settings) of the new policy we are training
- θ — the parameters of the old policy we collected data with
- $\pi_{\theta'}(a_{t,i} | s_{t,i})$ — probability the new policy picks action a in situation s
- $\pi_{\theta}(a_{t,i} | s_{t,i})$ — probability the old policy picks the same action
- $\hat{A}^{\pi_{\theta}}(s_{t,i}, a_{t,i})$ — the advantage: how much better this action was compared to average
- $\sum_{t,i}$ — add this up across all time steps t and all collected episodes i

What this equation is saying: We collected experience using the old policy θ , but we want to improve our new policy θ' without collecting new data each time. The fraction $\frac{\pi_{\theta'}}{\pi_{\theta}}$ is called the importance weight — it corrects for the fact that data was collected under a different policy. If the new policy would have chosen the same action even more often, this ratio is big; if less often, it is small. We multiply this ratio by the advantage $\hat{A}$, which tells us whether that action was actually good or bad. If an action was good (positive advantage) AND the new policy would do it more (ratio > 1), the score goes up. If the new policy has drifted far from the old one, the ratio explodes to huge or tiny numbers — that is the problem this equation alone cannot fix, which is why we need the tricks.

Trick #1 — Clip the importance weights

$$\tilde{J}(\theta') \approx \sum_{t,i} \text{clip}\left(\frac{\pi_{\theta'}(a_{t,i} \mid s_{t,i})}{\pi_{\theta}(a_{t,i} \mid s_{t,i})}, 1 - \epsilon, 1 + \epsilon\right) \hat{A}^{\pi_{\theta}}(s_{t,i}, a_{t,i})$$

New symbols:

- $\text{clip}(\cdot, 1 - \epsilon, 1 + \epsilon)$ — a safety clamp: if the value goes below $1 - \epsilon$, force it to $1 - \epsilon$; if it exceeds $1 + \epsilon$, force it to $1 + \epsilon$
- ϵ (epsilon) — a small number (typically 0.1 or 0.2) that sets how far the ratio is allowed to stray from 1

What this equation is saying: The importance weight ratio can get dangerously large or small when the new and old policies differ a lot, making training unstable. Clipping simply forces that ratio to stay inside the window $[1 - \epsilon, 1 + \epsilon]$ — for example $[0.8, 1.2]$ if $\epsilon = 0.2$. If the new policy has already changed too much from the old one, we stop rewarding it for changing further. It is like saying “you have wandered far enough from where you started — no extra credit for going further.” This keeps training stable.

Trick #2 — Take the minimum

$$\tilde{J}(\theta') \approx \sum_{t,i} \min\left(\frac{\pi_{\theta'}}{\pi_{\theta}} \hat{A}, \text{clip}\left(\frac{\pi_{\theta'}}{\pi_{\theta}}, 1 - \epsilon, 1 + \epsilon\right) \hat{A}\right)$$

New symbols:

- $\min(\cdot, \cdot)$ — take whichever of the two values is smaller

What this equation is saying: Trick #1 clipped the ratio, but that alone has a loophole: when the advantage is negative (meaning the action was bad), clipping might accidentally still give the policy an incentive to make changes. Taking the minimum of the clipped and unclipped objective plugs that loophole. Imagine two numbers: the “raw” score and the “clamped” score. We always take the worse one. This makes PPO pessimistic by design — it never rewards the policy for making a change that is not clearly safe and beneficial. This is the core PPO objective.

Page 13 — Trick #3: Generalized Advantage Estimation (GAE)

N-step Advantage

$$\hat{A}_n^{\pi}(s_t, a_t) = \sum_{t'=t}^{t+n} \gamma^{t'-t} r(s_{t'}, a_{t'}) - \hat{V}_{\phi}^{\pi}(s_t) + \gamma^n \hat{V}_{\phi}^{\pi}(s_{t+n})$$

What each symbol means:

- $\hat{A}_n^{\pi}(s_t, a_t)$ — the estimated advantage of taking action a_t at state s_t , looking n steps ahead
- $\sum_{t'=t}^{t+n}$ — sum rewards from now (t) up to n steps in the future

- $\gamma^{t'-t}$ — discount factor: rewards further in the future count less (closer = worth more)
- $r(s_{t'}, a_{t'})$ — actual reward received at each step
- $\hat{V}_\phi^\pi(s_t)$ — the critic’s estimated value of the current state (what it expected)
- $\gamma^n \hat{V}_\phi^\pi(s_{t+n})$ — the discounted value estimate at the state n steps later (a bootstrapped guess of what comes after)

What this equation is saying: Instead of looking either one step ahead (fast but inaccurate) or all the way to the end (accurate but noisy), this looks exactly n steps ahead. You sum up the real rewards you collected for n steps, then add the critic’s guess of what happens after step n (instead of running all the way to the end). Subtracting $\hat{V}(s_t)$ turns this from a raw return into an advantage — how much better or worse things went compared to what the critic expected. Small n = lower variance but possibly biased; large n = more accurate but noisier. GAE blends all values of n together.

GAE (Generalized Advantage Estimation)

$$\hat{A}_{\text{GAE}}^\pi(s_t, a_t) = \sum_{n=1}^{\infty} w_n \hat{A}_n^\pi(s_t, a_t), \quad w_n \propto \lambda^{n-1}$$

What each symbol means:

- $\hat{A}_{\text{GAE}}^\pi$ — the final blended advantage estimate
- $\sum_{n=1}^{\infty}$ — add up the advantage for every possible horizon: 1-step, 2-step, 3-step... all the way out
- w_n — the weight (importance) given to the n -step advantage
- λ (lambda) — a number between 0 and 1 that controls how fast the weights shrink
- λ^{n-1} — means 1-step gets weight 1, 2-step gets weight λ , 3-step gets λ^2 , etc.

What this equation is saying: Instead of committing to one fixed horizon n , GAE takes a weighted average of all horizons at once. The 1-step advantage gets the biggest weight, the 2-step gets slightly less, the 3-step even less, and so on — shrinking by a factor of λ each time. When $\lambda = 0$, only 1-step matters (low variance, more bias). When $\lambda = 1$, all horizons matter equally (low bias, high variance). You tune λ to get the best of both worlds. In practice $N \ll T$ (you do not actually go to infinity — you cut off early), and cutting earlier reduces variance at the cost of a little bias.

Page 14 — The Full PPO Algorithm

1. **Collect data:** Run the current policy π_θ in the environment to gather many episodes.
2. **Fit critic:** Train the value network $\hat{V}_\phi^\pi$ to match the observed summed rewards.
3. **Compute GAE advantage:** Use the n -step formula with weights $w_n \propto \lambda^{n-1}$ to get $\hat{A}_{\text{GAE}}^\pi$ for every action taken.

4. **Update policy:** Run M gradient steps (not just one!) on the full clipped and min surrogate objective from page 13.

The key payoff of all this machinery is Step 4: because PPO has safe clipping, you can squeeze out multiple gradient updates from a single batch of data without the policy flying off the rails. That is something vanilla policy gradients could not do — they had to collect new data after every single update.

The Big Picture Across All Three Pages

Think of it like a student who took one test (collected data under old policy θ). A naive student studies that test to death and overcorrects — completely changing how they think. PPO is the careful student: they use clipping to say “I will only shift my thinking so far from what I believed before the test,” use GAE to get smart credit assignment for each answer, and then review the same test paper multiple times safely — getting more learning out of each exam without going off the rails.

Give me a concrete example of trick 3 to explain it.

Concrete Example: GAE (Trick #3)

Scenario: A robot is learning to walk. At moment t , it steps forward. We want to know: was that step a good action?

Given numbers ($\gamma = 1$, no discounting, to keep it clean):

	Value
Reward at step t	+2 (moved forward)
Reward at step $t + 1$	+1 (another step)
Reward at step $t + 2$	−8 (fell down)
Critic’s guess for $V(s_t)$	3
Critic’s guess for $V(s_{t+1})$	4
Critic’s guess for $V(s_{t+2})$	5
Critic’s guess for $V(s_{t+3})$	0

Step 1 — Compute each n -step advantage

1-step (looks only 1 step ahead):

$$A_1 = r_t + V(s_{t+1}) - V(s_t) = 2 + 4 - 3 = +3$$

“Stepped forward, got +2, next state looks good. Action seems great!”

2-step (looks 2 steps ahead):

$$A_2 = r_t + r_{t+1} + V(s_{t+2}) - V(s_t) = 2 + 1 + 5 - 3 = +5$$

“Two good steps, state ahead looks even better. Looks amazing!” — but the fall is not visible yet.

3-step (looks 3 steps ahead):

$$A_3 = r_t + r_{t+1} + r_{t+2} + V(s_{t+3}) - V(s_t) = 2 + 1 + (-8) + 0 - 3 = -8$$

“Now we can see the robot falls at step $t + 2$. The action looks terrible.”

Step 2 — GAE blends all three with λ -weights

Weights: $w_1 = 1$, $w_2 = \lambda$, $w_3 = \lambda^2$

$$A_{\text{GAE}} = 1 \cdot A_1 + \lambda \cdot A_2 + \lambda^2 \cdot A_3$$

λ value	Meaning	Calculation	Result
$\lambda = 0$	Only 1-step	$1(+3)$	+3 — blind to the fall
$\lambda = 0.5$	Balanced	$1(+3) + 0.5(+5) + 0.25(-8)$	+3.5 — fall partially visible
$\lambda = 0.9$	Looks far ahead	$1(+3) + 0.9(+5) + 0.81(-8)$	+1.02 — fall heavily weighted
$\lambda = 1$	All equal weight	$1(+3) + 1(+5) + 1(-8)$	0 — fall fully cancels the gains

The punchline

The 1-step alone would tell the robot “stepping forward was a great move (+3)!” — because it cannot see the fall that happens two steps later.

The 3-step alone is too noisy — it blames the entire step forward harshly (−8) even though the first two steps were fine.

GAE with $\lambda = 0.9$ says +1.02 — which is honest: the step forward was slightly good, but not as good as it looked short-term, because trouble is coming. That is exactly the kind of precise, blame-accurate signal the actor needs to learn well.

Page 15 — The Big Question: Can We Reuse Old Data?

So far, the actor-critic algorithm only learns from data it just collected. Once you are done with a batch of experience, you throw it away. That is wasteful. Page 15 asks: What if we saved every single experience the agent ever had, and learned from ALL of it — not just the latest batch?

Two key ideas introduced:

1. **Replay Buffer** — a giant diary that stores every (state, action, reward, next-state) experience the agent has ever seen.
2. **Fix the equations** — the original equations assume you only use fresh data. If you use old data, you need to adjust the math.

Page 16 — First Attempt: Plug the Replay Buffer In

They take the standard algorithm and simply say: instead of doing steps on one fresh batch, do them on a small random sample (“minibatch”) pulled from the replay buffer — which contains all past experience.

The 5-step algorithm (first attempt):

1. Collect fresh experience and add it to the replay buffer.
2. Fit the value function $\hat{V}_\phi^\pi$ to reward sums from the buffer.

3. Calculate how good each (state, action) was using the advantage:

$$\hat{A}^\pi(s_i, a_i) = r(s_i, a_i) + \hat{V}_\phi^\pi(s'_i) - \hat{V}_\phi^\pi(s_i)$$

4. Estimate the policy gradient.
5. Update the policy using that gradient.

Steps 2–4 are done on a minibatch randomly sampled from all previous data — that is the key change.

Page 17 — Two Things Are Broken

Once they lay out the algorithm, they realize there is a serious problem. When V^π is fit on ALL data from the replay buffer (which comes from many different past versions of the policy), the value function represents some kind of average over old policies — not the current policy. This is a problem because the value function is supposed to tell you how good the *current* policy is.

Two things broken:

- **The target is wrong.** The value estimate $y = r + \gamma \hat{V}(s')$ uses a $\hat{V}$ trained on a mix of old policies, so it does not correctly represent the current policy's future rewards.
- **The likelihood is wrong.** In the gradient step, you are using the log probability of actions that old versions of the policy took — not actions the current policy would take.

Page 18 — The Fix: Use $Q(s, a)$ Instead of $V(s)$

Core insight: Instead of estimating how good a state is (V), estimate how good a state + action pair is (Q). This matters because $Q(s, a)$ can be computed from any experience, regardless of which policy collected it.

The equation:

$$Q^{\pi_\theta}(s, a) = r(s, a) + \gamma \mathbb{E}_{\substack{s' \sim p(\cdot | s, a) \\ \bar{a}' \sim \pi_\theta(\cdot | s')}} [Q^{\pi_\theta}(s', \bar{a}')]]$$

Symbol table:

Symbol	What it means
$Q^{\pi_\theta}(s, a)$	Q-value: expected total future reward starting in state s , taking action a , then following policy π_θ
$r(s, a)$	Immediate reward for taking action a in state s
γ	Discount factor (0 to 1); sooner rewards are worth more than later ones
$\mathbb{E}$	Expected value — average over many possible outcomes
$s' \sim p(\cdot \mid s, a)$	Next state sampled from the environment's transition function
$\bar{a}' \sim \pi_\theta(\cdot \mid s')$	Next action sampled from the <i>current</i> policy at next state s'
$Q^{\pi_\theta}(s', \bar{a}')$	Q-value of the next state-action pair

The quality of taking action a in state s equals the immediate reward right now, plus the discounted quality of whatever state and action comes next. The “next action” $\bar{a}'$ must be sampled from the *current* policy — not retrieved from the replay buffer — because we want to know how good the current policy is, not some past version. The approach: sample (s, a, s') from the replay buffer (old data is fine), but sample $\bar{a}'$ fresh from the current policy. Then the target becomes $y = r(s, a) + \gamma Q(s', \bar{a}')$, and you train the Q-network toward this target.

Page 19 — Fixing the Value Function: The Updated Algorithm

This page rewrites the algorithm with the fix from page 18 applied.

The key fix in step 3:

- **Old (broken):** Update $\hat{V}_\phi^\pi$ using targets $y_i = r_i + \gamma \hat{V}_\phi^\pi(s'_i)$
- **New (fixed):** Update $\hat{Q}_\phi^\pi$ using targets $y_i = r(s_i, a_i) + \gamma \hat{Q}_\phi^\pi(s'_i, a'_i)$, where $a'_i \sim \pi_\theta(\cdot \mid s'_i)$ — sampled from the *current* policy, not from the replay buffer.

Using a fresh action fixes the broken target problem — the Q-value target now reflects the current policy. The states s_i can still come from the replay buffer; only the next actions need to be fresh.

Page 20 — Two More Refinements

Refinement 1: Drop the baseline, just use Q directly.

Instead of computing the advantage $\hat{A}(s, a) = Q(s, a) - V(s)$, just use $Q(s, a)$ directly. This has higher variance (noisier estimates), but since you now have a huge replay buffer full of data, the extra noise is acceptable — more data cancels it out.

Refinement 2: Sample fresh actions for the gradient step too.

$$\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_i \nabla_\theta \log \pi_\theta(\bar{a}_i \mid s_i) \hat{Q}^\pi(s_i, \bar{a}_i)$$

Symbol table:

Symbol	What it means
$\nabla_{\theta} J(\theta)$	Gradient: the direction to nudge policy parameters θ to improve total reward
$\frac{1}{N} \sum_i$	Average over N sampled examples
$\nabla_{\theta} \log \pi_{\theta}(\bar{a}_i \mid s_i)$	How much the log-probability of action $\bar{a}_i$ changes when you tweak θ
$\bar{a}_i$	Fresh action sampled from current policy π_{θ} at state s_i — NOT the old stored action
$\hat{Q}^{\pi}(s_i, \bar{a}_i)$	Estimated quality of taking fresh action $\bar{a}_i$ in state s_i
$\pi_{\theta}(\bar{a}_i \mid s_i)$	Probability the current policy assigns to action $\bar{a}_i$ given state s_i

To improve the policy, look at a bunch of state-action pairs. For each state s_i from the buffer, sample a fresh action $\bar{a}_i$ from the current policy (not the old stored action). Check how good that fresh action is according to $\hat{Q}$. Then nudge the policy parameters in the direction that makes good actions more likely. Using a fresh action fixes the “wrong likelihood” problem from page 17 — you are now computing log probabilities under the current policy, not some old one. The states s_i can still come from the replay buffer; only the actions need to be fresh.

Page 21 — The Final Clean Algorithm

This is the polished, fixed version of the off-policy actor-critic with a replay buffer.

Final 5-step algorithm:

1. Collect experience $\{s_i, a_i\}$ from the current policy and add it to the replay buffer.
2. Sample a batch $\{s_i, a_i, r_i, s'_i\}$ from the replay buffer.
3. Update $\hat{Q}_{\phi}^{\pi}$ toward targets $y_i = r(s_i, a_i) + \gamma \hat{Q}_{\phi}^{\pi}(s'_i, a'_i)$, where $a'_i \sim \pi_{\theta}(\cdot \mid s'_i)$ — sampled fresh from the current policy.
4. Compute the gradient:

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_i \nabla_{\theta} \log \pi_{\theta}(\bar{a}_i \mid s_i) \hat{Q}^{\pi}(s_i, \bar{a}_i)$$

where $\bar{a}_i \sim \pi_{\theta}(\cdot \mid s_i)$ — freshly sampled from the current policy.

5. Update $\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$.

The replay buffer lets the agent learn from every experience it has ever had, making training much more data-efficient. The two key fixes — sampling fresh actions for the Q-target and for the gradient — ensure that even though the *data* is old, the *evaluation* always reflects what the current policy would do.

What is meant by random sample (minibatch)?

The replay buffer is like a giant jar full of thousands of past experiences (state, action, reward, next-state). Instead of learning from every single one of those experiences at once (too slow and memory-heavy), you randomly grab a small handful — say, 64 or 256 — and learn from just those.

That small random handful is the **minibatch**.

“Random” is important here: you do not pick the most recent ones or the best ones — you grab them randomly so the batch is not biased toward any one policy or time period. This also breaks correlations between nearby experiences, which helps the learning stay stable.

$Q(s,a)$ can be computed from any experience. How and why?

First, recall what each one asks:

- $V(s)$: “How good is this state, if I follow my current policy from here?” — depends entirely on the policy.
- $Q(s,a)$: “How good is it to take action a in state s , and then follow my current policy from there?” — the action is already pinned down.

Why Q can use old data:

The target for Q is:

$$y = r(s, a) + \gamma \hat{Q}^\pi(s', \bar{a}')$$

To compute this, you need three things:

Thing needed	Where it comes from	Policy-dependent?
$r(s, a)$	Replay buffer (old data)	No — the environment gave this reward, not the policy
s'	Replay buffer (old data)	No — the environment transitioned to s' , not the policy
$\bar{a}'$	Sampled fresh from current policy	Yes — and you do this fresh

The reward $r(s, a)$ is a fact about the environment — it does not matter which policy decided to take action a . If you step off a cliff in a game, you lose a life whether your old policy did it or your new one. The environment always gives the same reward for the same (s, a) . The next state s' is also determined by the environment, not the policy. So the (s, a, r, s') tuple from old experience is completely valid. The only policy-sensitive piece — what action comes next — is handled by sampling $\bar{a}'$ fresh from the current policy at s' .

Why V cannot use old data:

$V(s)$ has no action pinned. It averages over whatever actions the policy would take. So if your replay buffer has data from 10 different old policies, $V(s)$ ends up representing some confused blend of all of them — not your current policy. There is no way to “correct” for this without knowing exactly which old policy produced each experience.

In short: Q separates the policy-independent part (r, s') from the policy-dependent part ($\bar{a}'$). Old data handles the first part; the current policy handles the second. V has no such clean separation.

3 Imitation Learning

Pages 2–6 — Imitation Learning: Problem Setup Through Expressive Policies

Page 2 — Imitation Learning: Problem Setup

The core idea: we are given a pile of recorded examples of an expert doing a task well (states paired with the actions the expert took), and we want a policy that acts just as well by copying those examples, without ever being told a reward signal or being told the rules of “good” behavior directly.

Mechanism

The data is a set of trajectories

$$\mathcal{D} = \{(s_1, a_1, \dots, s_T)\},$$

where each trajectory is assumed to be a sample from an unknown expert policy π_{expert} .

Symbol	What it means
s_t	the state (or observation) at time step t
a_t	the action taken at time step t
T	the length of one demonstrated trajectory
$\mathcal{D}$	the whole collected dataset of expert trajectories
π_{expert}	the (unknown) rule the expert used to pick actions
π_θ	the policy we are training, with learnable parameters θ

We never see π_{expert} itself, only its footprints (the (s, a) pairs it left behind). The goal is to fit π_θ so that it performs at roughly the same level as the expert, by mimicking the patterns in $\mathcal{D}$, rather than by being told a reward function. The driving example on the slide makes this concrete: the state is sensor readings from a car, the action is a steering/acceleration command, and the dataset is simply recordings of human drivers.

Page 3 — Behavioral Cloning (“Imitation Learning — Version 0”)

The simplest possible approach: treat this exactly like an ordinary supervised-learning regression problem. For every state in the dataset, make the policy’s predicted action match the expert’s recorded action as closely as possible.

Mechanism

$$\min_{\theta} \frac{1}{|\mathcal{D}|} \sum_{(s,a) \in \mathcal{D}} \|a - \hat{a}\|^2, \quad \hat{a} = \pi_\theta(s)$$

Symbol	What it means
$ \mathcal{D} $	the number of (s, a) pairs in the dataset
a	the action the expert actually took in state s
$\hat{a}$	the action our deterministic policy predicts for that same s
$\pi_\theta(s)$	a deterministic function: one state in, one action out
$\ \cdot\ ^2$	squared Euclidean distance between the two action vectors

In plain terms: run the state through the network, get a predicted action, measure how far off it was from what the expert did, and nudge the network’s weights (via gradient descent) to shrink that distance, averaged over the whole dataset. Once trained, you simply run π_θ in the world — no further learning. This is behavioral cloning (BC) with a deterministic policy, and it is exactly ordinary L_2 regression from states to actions.

Page 4 — What Can Go Wrong? The Mean-Collapse Problem

If the expert is not perfectly consistent — different demonstrators solve the same situation in different, equally valid ways — L_2 regression does not pick one of those ways. It averages them, and the average can be a terrible, even unsafe, action.

Mechanism

Squared-error loss is minimized, for a fixed state s , by predicting the *mean* of the conditional action distribution $p(a \mid s)$ in the data. This is a basic fact about squared loss (the same reason the sample mean minimizes sum-of-squared-deviations), not something special to neural networks. So if $p(a \mid s)$ is multimodal — say, some experts merge left and others stay straight — L_2 regression will output the average of those two very different actions, which is a point that lies *between* the modes and can have almost zero probability under the real data.

Numerical Walkthrough

The slide’s demonstration-distribution plot shows two modes of steering command: “merge left” centered near -2.0 and “stay straight” centered near 0 , with staying straight the more common choice. Suppose, concretely, that 75% of the recorded examples are “stay straight” ($a \approx 0$) and 25% are “merge left” ($a \approx -2$):

$$\mathbb{E}[a \mid s] = 0.75 \times 0 + 0.25 \times (-2) = -0.5.$$

That matches the dashed “mean of data” line on the slide, sitting at about -0.5 — roughly halfway between the lane markings, straddling both lanes, which is exactly the unsafe, low-probability action the slide warns about. The lesson: *we need to learn beyond the mean*, i.e., we need the policy to represent a distribution that can put probability mass separately on -2 and on 0 , rather than collapsing them into one number.

Page 5 — Neural Networks as Distribution Parameterizations

A neural network that outputs a single action vector can only ever be as expressive as whatever distribution class you decided to wrap around it. Making the network bigger does not fix this — expressive power of the *function* $s \rightarrow$ parameters is a separate question from expressive power of the *distribution class* those parameters define.

Mechanism

Instead of directly outputting an action, the network outputs the parameters of a distribution over actions, $p(a \mid s)$:

- **1D discrete actions** (e.g., Mario: up, down, press A, press B): the network outputs one probability per action, i.e., a full categorical distribution $p(\text{up}), p(\text{down}), \dots$. This is *maximally expressive* for discrete actions — there is no distribution over a finite set of options that a categorical cannot represent exactly.
- **Continuous actions** (e.g., steering angle): the simplest choice is a single Gaussian, so the network outputs $\mu(s)$ and $\sigma(s)$. This is *not very expressive* — a single Gaussian is always unimodal, so it can never represent the two-mode “merge left / stay straight” situation from Page 4 no matter how large the network is.

Contrast to avoid confusing: the expressive power of the mapping $s \rightarrow (\mu, \sigma)$ (which can be made arbitrarily flexible by using a bigger network) is not the same thing as the expressive power of the Gaussian distribution itself (which is fixed and unimodal regardless of network size). A giant network can learn a very accurate $\mu(s)$, but it still cannot describe two separate peaks with only one Gaussian.

Page 6 — Imitation Learning — Version 1: Expressive Policies

The fix for Page 4’s failure mode: instead of regressing to the mean action, train a generative model that assigns high probability to whatever action the expert actually took, using a distribution class expressive enough to have multiple modes.

Mechanism

$$\min_{\theta} -\mathbb{E}_{(s,a) \in \mathcal{D}} [\log \pi_{\theta}(a \mid s)]$$

Symbol	What it means
$\pi_{\theta}(a \mid s)$	the probability (density) our policy assigns to action a in state s
$\log \pi_{\theta}(a \mid s)$	log-likelihood of the expert’s actual action under our policy
$-\mathbb{E}_{(s,a) \in \mathcal{D}} [\cdot]$	average negative log-likelihood over the dataset

This is maximum likelihood estimation: we want π_{θ} to assign as much probability as possible to the actions the expert actually chose, for every (s, a) pair in the dataset. Minimizing the negative log-likelihood is the same optimization as maximizing the likelihood; the sign flip just turns “maximize” into a “minimize” so it fits the usual gradient-descent machinery. Deployment is unchanged: once trained, just sample actions from $\pi_{\theta}(\cdot \mid s)$ (or take its mode) and run the policy.

Pages 7, 8 — Multimodal Action Models: GMMs and Autoregressive Discretization

Page 7 — Gaussian Mixture Models (GMMs)

A GMM is the simplest fix for the “one Gaussian can’t be multimodal” problem: instead of one bump, let the network output several weighted bumps, so it can place separate

probability mass on “merge left” and “stay straight” at once.

Mechanism

The network takes state s and outputs the parameters of K Gaussian components:

$$\{w_k(s), \mu_k(s), \Sigma_k(s)\}_{k=1}^K.$$

The probability of an action is the weighted sum of the K component densities:

$$p(a_i | s_i) = \sum_{k=1}^K w_k(s_i) \mathcal{N}(a_i | \mu_k(s_i), \Sigma_k(s_i)).$$

Training maximizes the total log-likelihood of the demonstrated actions, equivalently minimizing the negative log-likelihood:

$$\mathcal{L}(\theta) = - \sum_{i=1}^N \log \left[\sum_{k=1}^K w_k(s_i) \mathcal{N}(a_i | \mu_k(s_i), \Sigma_k(s_i)) \right].$$

Symbol	What it means
K	number of Gaussian components (a hyperparameter you choose ahead of time)
$w_k(s)$	mixture weight of component k in state s ; all $w_k \geq 0$, $\sum_k w_k = 1$
$\mu_k(s)$	mean (center) of component k ’s Gaussian bump
$\Sigma_k(s)$	covariance (spread) of component k ’s Gaussian bump
$\mathcal{N}(a \mu, \Sigma)$	the Gaussian probability density evaluated at a
N	number of (s, a) training pairs

Tensor shapes / parameter count. If the action is d -dimensional and each component has a diagonal covariance, the network’s output layer must produce K scalars for w_k , $K \times d$ numbers for the means, and $K \times d$ numbers for the (diagonal) variances — a total output vector of size $K(1 + 2d)$ (the weights are typically produced as raw scores and passed through a softmax over k so they sum to one and stay non-negative).

Numerical Walkthrough

Let $K = 2$, and suppose for a given state s the network outputs $w_1 = 0.6$, $\mu_1 = -2$, $\sigma_1 = 0.3$ (merge-left component) and $w_2 = 0.4$, $\mu_2 = 0$, $\sigma_2 = 0.3$ (stay-straight component), with 1-D actions. The 1-D Gaussian density is $\mathcal{N}(a | \mu, \sigma^2) = \frac{1}{\sigma\sqrt{2\pi}} e^{-(a-\mu)^2/(2\sigma^2)}$. Evaluate the mixture density at $a = -2$ (an exact merge-left action):

$$\mathcal{N}(-2 | -2, 0.3^2) = \frac{1}{0.3\sqrt{2\pi}} e^0 \approx 1.330, \quad \mathcal{N}(-2 | 0, 0.3^2) = \frac{1}{0.3\sqrt{2\pi}} e^{-\frac{4}{0.18}} \approx 1.330 \times e^{-22.2} \approx 0.$$

So $p(a = -2 | s) \approx 0.6(1.330) + 0.4(0) = 0.798$ — almost all the probability mass at $a = -2$ comes from component 1, exactly as intended: a GMM can assign high probability to -2 and (separately, via component 2) high probability to 0, something a single Gaussian could never do simultaneously.

Page 8 — Multimodal Actions via Autoregressive Models (Discretize + Chain Rule)

When the action itself has several dimensions (e.g., steering *and* acceleration), a GMM over the whole action vector can get unwieldy. The autoregressive trick: predict one dimension at a time, each time conditioning on the dimensions already chosen, using the chain rule of probability so nothing is lost.

Mechanism

For a multidimensional action $a_i = (a_{i,1}, a_{i,2}, \dots, a_{i,D})$, the chain rule of probability lets us write the exact joint distribution as a product of conditionals:

$$p_\theta(a_i | s_i) = \prod_{d=1}^D p_\theta(a_{i,d} | s_i, a_{i,1:d-1}).$$

Training maximizes the likelihood of the demonstrated actions, i.e., minimizes

$$\mathcal{L}(\theta) = - \sum_{i=1}^N \sum_{d=1}^D \log p_\theta(a_{i,d} | s_i, a_{i,1:d-1}).$$

Symbol	What it means
D	number of dimensions in the action (e.g., $D = 2$: steering, acceleration)
$a_{i,1:d-1}$	the first $d - 1$ action dimensions already chosen, for example i
$p_\theta(a_{i,d} s_i, a_{i,1:d-1})$	probability of dimension d given the state and prior dimensions

Each per-dimension conditional is typically represented by *discretizing* that one dimension into bins and outputting a categorical distribution over bins (a classification problem), which is why this is called “discretize + autoregressive.” This is exact — the chain rule is an identity, not an approximation — and the ordering of dimensions is a design choice (a hyperparameter) that does not change how much the model can represent in principle, only how efficiently it does so.

Numerical Walkthrough

Say $D = 2$: dimension 1 is steering, discretized into 3 bins {left, straight, right}, and dimension 2 is acceleration, discretized into 2 bins {low, high}. Suppose the network outputs $p(\text{left}) = 0.3$, $p(\text{straight}) = 0.6$, $p(\text{right}) = 0.1$ for dimension 1. If the sampled/true first action is “left,” then dimension 2’s distribution is recomputed *conditioned on that*, e.g., $p(\text{high} | \text{left}) = 0.8$, $p(\text{low} | \text{left}) = 0.2$ (turning left into oncoming traffic requires more acceleration to clear the gap). The joint probability of the pair (left, high) is then $p(\text{left}) \times p(\text{high} | \text{left}) = 0.3 \times 0.8 = 0.24$ — a single number obtained by multiplying the two conditionals, exactly as the chain-rule formula specifies. Note this joint probability is different from what it would have been if we (incorrectly) assumed independence using the *unconditional* marginal for acceleration, which is why conditioning on prior dimensions matters whenever the dimensions of the action are correlated (as steering and acceleration are here).

Page 9 — Autoregressive Models: Sequential Execution

Page 9 — Autoregressive Models: Sequential Execution

This slide is the “how do you actually run it” companion to Page 8: the same factorization, but written out as a step-by-step sampling procedure, exactly analogous to how a language model generates a sentence one word at a time.

Mechanism

$$\pi_{\theta}(a \mid s) = \prod_{d=1}^D \pi_{\theta}(a_d \mid s, a_{<d}), \quad a_{<d} = (a_1, \dots, a_{d-1}).$$

At execution (inference) time, the dimensions are generated one at a time, each sampled from a distribution conditioned on everything sampled so far:

$$a_1 \sim \pi_{\theta}(a_1 \mid s), \quad a_2 \sim \pi_{\theta}(a_2 \mid s, a_1), \quad \dots, \quad a_D \sim \pi_{\theta}(a_D \mid s, a_{1:D-1}).$$

So the complete action is produced by the chain $s \rightarrow a_1 \rightarrow a_2 \rightarrow \dots \rightarrow a_D$: each newly sampled dimension is fed back into the model as input for predicting the next dimension. This is structurally identical to a language model predicting one token, feeding that token back in, and predicting the next token — the only difference is that “tokens” here are action-dimension bins rather than words. Because the factorization is exact, this autoregressive policy loses nothing in principle relative to modeling the full joint $\pi_{\theta}(a \mid s)$ directly; it only trades a hard joint-probability problem for a sequence of easier one-dimension-at-a-time problems.

Page 10 — Decision Transformer

Page 10 — Decision Transformer

The slide is very terse here (just a diagram and a citation), so this section leans on the original paper, Chen et al., *Decision Transformer: Reinforcement Learning via Sequence Modeling*, NeurIPS 2021 (arXiv:2106.01345), Sections 2–3.

Core Idea

Instead of fitting a value function or computing policy gradients, treat control as a sequence-modeling problem: feed a causally-masked (GPT-style) transformer a sequence of *(return-to-go, state, action)* triples and train it to predict the next action, the same way a language model predicts the next word.

Mechanism

- **Return-to-go** $\hat{R}_t = \sum_{t'=t}^T r_{t'}$: the sum of *future* rewards from time t onward. This is fed in as an explicit input token, not just used as a training label.
- **Tokenization**. At each timestep t , three raw inputs — $\hat{R}_t, s_t, a_t$ — are each passed through their own modality-specific linear embedding layer (one small learned matrix per modality) to become vectors of the same width d_{model} , and a learned positional (timestep) encoding is added to each.

- **Sequence.** These embedded tokens are concatenated in order $(\hat{R}_1, s_1, a_1, \hat{R}_2, s_2, a_2, \dots)$, so a trajectory of length T becomes a sequence of $3T$ tokens.
- **Causal transformer.** The sequence is passed through a GPT-style transformer with a causal attention mask, meaning each token can only attend to itself and earlier tokens, never later ones — this is what makes autoregressive generation well-defined.
- **Linear decoder.** At the position of each state token, a linear head reads the transformer’s output and predicts the corresponding action.

Symbol	What it means
$\hat{R}_t$	return-to-go at time t : total reward the trajectory earns from t onward
d_{model}	width of the embedding vectors fed to the transformer
causal mask	attention pattern that blocks a token from seeing future tokens

Tensor shapes (toy example). Suppose $d_{\text{model}} = 4$ and a trajectory has $T = 3$ timesteps. Each of the three modalities (return, state, action) gets its own embedding matrix mapping its raw dimensionality into $\mathbb{R}^4$; after embedding, every token is a length-4 vector, and the full input to the transformer is a sequence of $3T = 9$ such vectors, i.e., a 9×4 matrix (plus positional encodings of the same shape, added elementwise). The linear decoder is a single $4 \times |\mathcal{A}|$ (or $4 \times d_a$ for continuous actions) weight matrix applied at each state-token position.

Why this matters at test time: because return-to-go is an explicit *input*, you can condition generation on a high desired return (e.g., “pretend the rest of this trajectory will earn +100”) and the model, having learned the correlation between stated return-to-go and the actions that achieve it, outputs actions consistent with a high-performing trajectory — effectively turning a hindsight-labeled offline dataset into a controllable, prompt-able policy, entirely through supervised sequence prediction rather than Bellman backups.

Page 11 — “Diffusion Policy” (Flow-Matching Formulation)

Page 11 — “Diffusion Policy” (Flow-Matching Formulation)

Flag/caveat. The slide labels this section “Diffusion Policy,” but the exact training and inference recipe shown — linear interpolation between noise and data, and regressing a velocity field onto $(x_1 - x_0)$ — is the *flow matching* / rectified-flow formulation (Lipman et al., *Flow Matching for Generative Modeling*, 2022), not the original Diffusion Policy paper’s recipe. The original Diffusion Policy (Chi et al., RSS 2023, arXiv:2303.04137) uses a DDPM-style noise-prediction objective with many discrete noise levels $x^k = \sqrt{\bar{\alpha}_k} x^0 + \sqrt{1 - \bar{\alpha}_k} \epsilon$ and trains a network to predict the injected noise ϵ . Flow matching instead uses a straight-line path between a noise sample and a data sample and regresses directly onto a constant velocity along that line. Both are “generate an action by transforming noise” methods and both are used under the umbrella term “diffusion policy” in the robotics literature, but they are different training objectives; several recent large-scale robot policies (e.g., Physical Intelligence’s π_0) use exactly the flow-matching recipe shown on this slide rather than the original DDPM recipe.

Mechanism

Training

1. Sample a real action $x_1 \sim \mathcal{D}$ and a noise sample $x_0 \sim \mathcal{N}(0, 1)$.
2. Sample a time $t \sim p(t)$, $t \in [0, 1]$.
3. Linearly interpolate: $x_t = tx_1 + (1 - t)x_0$.
4. Minimize $\|v_\theta(x_t, t) - (x_1 - x_0)\|^2$.

Inference

1. Sample noise $x_0 \sim \mathcal{N}(0, 1)$.
2. For $t \in \{0, \delta, 2\delta, \dots, 1 - \delta\}$: $x_{t+\delta} = x_t + v_\theta(x_t, t) \delta$ (an Euler step).
3. Return x_1 .

Symbol	What it means
x_1	a real action sampled from the demonstration data
x_0	a sample of pure Gaussian noise, same shape as an action
t	interpolation time, 0 = pure noise, 1 = real data
x_t	the point on the straight line between x_0 and x_1 at time t
$v_\theta(x_t, t)$	the network's guess of the velocity (direction and speed) at x_t
δ	the step size used to numerically integrate the ODE at inference

The key insight: along a straight-line path from x_0 to x_1 , the true instantaneous velocity is simply the constant vector $x_1 - x_0$ (the displacement, since position moves at a constant rate along a straight line). So training just means: pick a random point along the line, and teach the network to predict which straight line (i.e., which endpoint pair) that point came from, using only its slope. At inference we don't know x_1 , so we start from noise x_0 and repeatedly ask the network “which direction should I move,” taking small steps — this numerically solves the differential equation $dx/dt = v_\theta(x, t)$ forward from $t = 0$ to $t = 1$. For imitation learning, the whole process is additionally conditioned on the state s_t (i.e., $v_\theta(x_t, t, s_t)$), so different states can produce different denoising paths and hence different, possibly multimodal, action outputs.

Numerical Walkthrough

Let the action be 1-D. Suppose $x_1 = 2$ (expert action) and $x_0 = -1$ (a sampled noise value), and we train at $t = 0.3$:

$$x_t = 0.3(2) + 0.7(-1) = 0.6 - 0.7 = -0.1, \quad \text{target velocity} = x_1 - x_0 = 2 - (-1) = 3.$$

So the single training example teaches the network: “if you see the point -0.1 at time 0.3 , the correct velocity to output is 3 .” At inference, starting from $x_0 = -1$ with, say, two Euler steps of size $\delta = 0.5$, and supposing the trained network (approximately) outputs the constant velocity 3 at every point on this path:

$$x_{0.5} = x_0 + v_\theta(x_0, 0) \cdot 0.5 = -1 + 3(0.5) = 0.5, \quad x_1 = x_{0.5} + v_\theta(x_{0.5}, 0.5) \cdot 0.5 = 0.5 + 3(0.5) = 2.0,$$

recovering $x_1 = 2$ exactly, because the path really is a straight line with constant velocity 3 in this simplified illustration (with a real trained network the velocity estimate is only approximate, and using more, smaller steps reduces the resulting numerical error).

Pages 12, 13 — Compounding Errors and DAgger

Page 12 — Problems with Imitation Learning: Compounding Errors

Ordinary supervised learning assumes the inputs you’re tested on come from the same fixed distribution as your training inputs. Imitation learning breaks this assumption: the policy’s own actions determine which states it visits next, so a small mistake changes the input distribution the policy will face going forward.

Mechanism

The diagram shows a demonstrated trajectory $s_1 \rightarrow s_2 \rightarrow s_3 \rightarrow s_4$ (teal dots) versus what happens when the learned policy makes small errors $\hat{a}_1, \hat{a}_2, \hat{a}_3$ at each step: the very first small error nudges the policy to a state slightly off the demonstrated path; because that off-path state was underrepresented (or absent) in the training data, the policy is now *more* likely to err there too, nudging it even further off; the error keeps growing. This is called **compounding error**, and the resulting phenomenon — the states visited by the learned policy $p_\pi(s)$ differing from the states visited by the expert $p^*(s)$ — is called **covariate shift**.

Reference expansion (class transcript, imitation_learning.md, “Compounding Errors” section): the transcript frames this precisely as a break from the i.i.d. assumption of supervised learning — “the inputs to the model are completely independent from the predictions the model makes” in ordinary supervised learning, but “this all changes in imitation learning” because actions drive state transitions. It also notes that even a policy that fits the training distribution *perfectly* can still drift once it makes any small mistake, because perfect in-distribution accuracy says nothing about what happens once you leave the distribution.

Numerical Illustration of Why This Compounds

Suppose the policy has a small per-step probability $\epsilon = 0.05$ of making an error large enough to leave the expert’s state distribution, and once off-distribution it behaves essentially arbitrarily for the rest of a horizon of $T = 50$ steps. The chance of surviving all 50 steps without a single such error is

$$(1 - \epsilon)^T = (0.95)^{50} \approx 0.077,$$

i.e., only about a 7.7% chance of never drifting off. Ross and Bagnell formalized this precisely: for behavioral cloning with per-state error probability ϵ under the expert’s own state distribution, the gap between the expert’s total reward and the learned policy’s total reward can grow *quadratically* in the horizon, $J(\pi^*) - J(\pi) = O(\epsilon T^2)$ (Ross & Bagnell, *Efficient Reductions for Imitation Learning*, 2010). This quadratic-in- T growth is exactly the mathematical signature of compounding error, and it is what the corrective-data methods on the next page are designed to fix.

Page 13 — DAgger: Dataset Aggregation

If compounding error comes from the policy visiting states the expert never demonstrated, the natural fix is to get expert labels *at the states the policy actually visits* — not just the states the expert originally happened to visit.

Mechanism

1. Roll out the current learned policy π_θ to get a trajectory $s'_1, a'_1, \dots, s'_T$.
2. Query the expert for what it would have done at each visited state: $a^* \sim \pi_{\text{expert}}(\cdot \mid s')$.
3. Aggregate these corrections into the dataset: $\mathcal{D} \leftarrow \mathcal{D} \cup \{(s', a^*)\}$.
4. Update (retrain or fine-tune) the policy: $\min_\theta \mathcal{L}(\pi_\theta, \mathcal{D})$.

Repeat this loop: each round, the growing dataset $\mathcal{D}$ increasingly covers the states the *current* policy tends to visit (including its own mistakes), not just the states the original expert happened to visit.

Symbol	What it means
s'	a state visited by the current learned policy's rollout (possibly off-distribution)
a^*	the expert's action, specifically queried at that visited state s'
$\pi_{\text{expert}}(\cdot \mid s')$	the expert consulted live, not just replayed from old logs

Reference expansion (class transcript). The transcript notes a practical variant of step 2: instead of asking the expert to name the exact action for a given visited state (which can be very hard, e.g., “what steering command exactly?”), you can instead let the expert take full manual control once a mistake is noticed, and log that partial demonstration — this sidesteps having to introspect an exact corrective action. The transcript also flags an important caveat: corrective data must be collected across a *diverse* range of scenarios, or the aggregated dataset overfits to the narrow conditions under which corrections happened (e.g., correcting only on one campus, then failing to generalize elsewhere).

Why this fixes the compounding-error bound. Whereas naive behavior cloning's error grows as $O(\epsilon T^2)$ (Page 12), Ross, Gordon, and Bagnell showed DAgger's reduction to online no-regret learning achieves an error bound that is only *linear* in the horizon, $O(\epsilon T)$ (Ross, Gordon & Bagnell, *A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning*, AISTATS 2011) — directly because the training distribution now matches the distribution the deployed policy will actually encounter. The tradeoff is cost: DAgger needs a live, queryable expert during training, which is not always available or safe.

Pages 14–16 — Why Offline RL, Formal Setup, and Trajectory Stitching

Page 14 — Why Offline RL?

Having just seen that online correction (DAgger) fixes compounding error but requires an expert available live, this slide sets up the alternative: what if we instead squeeze more

out of a *fixed*, already-collected dataset, using reward labels instead of (or in addition to) an expert?

Mechanism

Online RL process	Offline RL process
Collect data	Given a static dataset
Update policy on the latest data (or all data so far)	Train the policy on the provided dataset only

Why offline RL can be attractive: it lets you reuse datasets already collected by people or by existing systems (previous experiments, robots, institutions) instead of paying to recollect; it avoids the risk of running an undertrained policy in the world to gather more data (unsafe in, e.g., medical or driving domains); and it doesn't require writing down a reward function during data collection, only afterward for training. A common hybrid is offline pretraining followed by online fine-tuning, combining a safe, cheap starting point with a smaller, safer amount of live interaction.

Page 15 — Offline RL: Formal Setup

This section restates the RL objective (maximize expected return) but makes the data-generating process explicit: the dataset now comes from an unknown, possibly suboptimal, possibly mixed *behavior policy*, and unlike imitation learning, we assume we also have reward labels r attached to every transition.

Mechanism

$$\mathcal{D} : \{(s, a, s', r)\} \sim \pi_\beta, \quad p_{\pi_\beta}(\tau) = p(s_1) \prod_{t=1}^T \pi_\beta(a_t | s_t) p(s_{t+1} | s_t, a_t),$$

$$\max_{\theta} \mathbb{E}_{p_{\pi_\theta}} \left[\sum_t r(s_t, a_t) \right].$$

Symbol	What it means
π_β	the (unknown) <i>behavior policy</i> that generated the dataset
$p_{\pi_\beta}(\tau)$	the probability of an entire trajectory τ under π_β and the environment dynamics
$p(s_1)$	the distribution of starting states
$p(s_{t+1} s_t, a_t)$	the environment's (unknown, fixed) transition dynamics
$r(s_t, a_t)$	the reward received for taking action a_t in state s_t

Two things are new relative to imitation learning: (1) we now have reward labels r , so “good” is defined numerically rather than purely by what the expert happened to do, and (2) π_β is explicitly allowed to be a *mixture* of policies (data from many different sources, of many different skill levels) rather than one consistent expert. The objective itself — maximize expected return under the policy we’re learning, π_θ — is the same objective as ordinary (online) RL; what changes is that we are only allowed to use $\mathcal{D}$, never new rollouts, to solve it.

Page 16 — Offline RL versus Imitation Learning: Trajectory Stitching

Even with reward labels available, why bother with RL machinery instead of just imitating the whole dataset? Because a dataset can contain *no single trajectory* that is good all the way through, while still containing all the pieces needed to construct one.

Mechanism

The diagram shows two overlapping recorded trajectories that pass through a shared state. One continuation from that shared state reaches a rewarding state s_9 (with $r(s_9) = +1$); a different continuation from the very same shared state instead wanders to a less useful state s_6 (with $r(s_6) = -1$). Neither full recorded trajectory, taken as a whole, is uniformly good — but the *first half* of one trajectory (getting to the shared state) combined with the *second half* of the other (continuing on to s_9) would be. Behavior cloning can only ever reproduce a trajectory it saw; it has no mechanism for splicing together the good part of one trajectory with the good part of another. Offline RL, by contrast, learns state-indexed values (via Bellman backups) rather than whole-trajectory imitation: at the shared state, whichever continuation has higher backed-up value gets favored regardless of which original trajectory it came from, effectively **stitching** the good first half to the good second half into a policy better than either recorded trajectory alone.

Reference expansion. This “stitching” property of value-based (Bellman-backup) methods versus trajectory-imitation methods is discussed explicitly in Levine, Kumar, Tucker & Fu, *Offline Reinforcement Learning: Tutorial, Review, and Perspectives on Open Problems* (arXiv:2005.01643, 2020): imitation-style methods are fundamentally limited by the return of the single best trajectory (or best sub-population) actually present in the data, whereas dynamic-programming-style offline RL methods can in principle exceed any individual trajectory’s return by recombining sub-trajectories through the state-value function. The bracketed questions on the slide point at this directly: a policy learned with vanilla imitation would just reproduce whichever recorded path it was shown (never reaching s_9 if that specific full path was never demonstrated), while a correctly-learned RL policy should be able to reach s_9 by stitching; whether **TD** or **Monte Carlo** value estimation is used matters here because MC value estimates are tied to the return of the specific trajectory they were sampled from, while TD backups propagate value *state-by-state* and are exactly the mechanism that makes cross-trajectory stitching possible.

Pages 17–19 — Why Not Off-Policy Directly, the OOD Action Problem, and Filtered BC

Page 17 — Why Not Just Run an Off-Policy Algorithm Directly?

Off-policy actor-critic methods like SAC already claim to work from a replay buffer of past data. So why not just point one at a fixed offline dataset and call it a day? This slide sets up the question; the next slide gives the answer.

Mechanism

The standard SAC-style Q-function update:

$$\min_{\phi} \sum_{(s,a,s') \sim \mathcal{D}} \left\| \hat{Q}_{\phi}^{\pi_{\theta}}(s,a) - (r(s,a) + \gamma \mathbb{E}_{a' \sim \pi_{\theta}(\cdot | s')} [\hat{Q}_{\phi}^{\pi_{\theta}}(s',a')]) \right\|^2,$$

followed by a policy update using the reparameterized (or likelihood-ratio) policy gradient

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_i \nabla_{\theta} \log \pi_{\theta}(a_i^{\pi^*} | s_i) \hat{Q}_{\phi}^{\pi_{\theta}}(s_i, a_i^{\pi^*}), \quad a_i^{\pi^*} \sim \pi_{\theta}(a | s_i).$$

Symbol	What it means
$\hat{Q}_{\phi}^{\pi_{\theta}}(s,a)$	the learned critic’s estimate of the value of taking a in s under π_{θ}
γ	discount factor
$a' \sim \pi_{\theta}(\cdot s')$	the <i>current</i> policy’s own action at the next state, used for bootstrapping
$a_i^{\pi^*}$	an action freshly sampled from the current policy, used to compute the policy gradient

The critical detail buried in this formula: both the Q-update’s bootstrap target and the policy-gradient step evaluate $\hat{Q}_{\phi}$ at an action a' (or $a_i^{\pi^*}$) sampled from the *current* policy π_{θ} , not necessarily an action that ever appears in the fixed dataset $\mathcal{D}$. When new data keeps arriving (the online setting), any inaccuracy this introduces gets corrected the next time that action is actually tried in the environment. With a *static* dataset from a mediocre policy, that corrective feedback loop is gone — which is exactly the problem the next page dissects.

Page 18 — The Out-of-Distribution (OOD) Action Problem

This is the central reason naive off-policy algorithms fail offline: the critic gets evaluated at actions it was never trained on, its errors there are never corrected by real experience, and those errors get amplified rather than fixed by the Bellman backup.

Mechanism

Consider the same critic loss as Page 17. At a next-state s' , the target requires $\mathbb{E}_{a' \sim \pi_{\theta}(\cdot | s')} [\hat{Q}_{\phi}^{\pi_{\theta}}(s',a')]$ — an average over actions the *current* policy would choose, which may fall well outside the region of actions actually present in $\mathcal{D}$ for that state (the “data support”). The diagram plots a randomly-initialized $Q(s',a')$ as a function of a' : it is a wiggly, essentially arbitrary curve outside the narrow green data-support region, since it was never trained there. If the policy update is free to pick whichever a' maximizes this untrained curve, it will latch onto one of the curve’s random-noise peaks lying *outside* the data support — an action the critic simply has no real information about.

Symbol	What it means
data support	the range of actions actually observed in $\mathcal{D}$ for a given state
OOD action	an action fed to Q that lies outside that observed range

This creates a feedback loop with no brake: (1) Q is unreliable (essentially noise) on OOD actions; (2) the policy update seeks out exactly those actions where Q happens to look over-optimistic; (3) the next Bellman backup then treats that already-inflated value as a real target and inflates Q further; (4) each round the learned policy drifts further from the behavior policy, chasing an ever more overestimated (and ever less trustworthy) Q -value. This phenomenon is documented in the offline RL literature as *extrapolation error* or the OOD-action overestimation problem, e.g., Fujimoto, Meger & Precup, *Off-Policy Deep Reinforcement Learning without Exploration* (BCQ, 2019) and Kumar, Zhou, Tucker & Levine, *Conservative Q-Learning for Offline Reinforcement Learning* (CQL, 2020).

Numerical Walkthrough

Suppose the true (unobservable) values for a state s' are $Q^*(s', a' = 1) = 5$ (in-support) and $Q^*(s', a' = 9) = 0$ (far out of support, a bad action). Before training converges, the randomly initialized network might output $\hat{Q}_\phi(s', 1) = 4.8$ (close to truth, since $a' = 1$ is well covered by data) but $\hat{Q}_\phi(s', 9) = 7.0$ (a spurious overestimate, since $a' = 9$ was never trained on). A policy update that samples candidate actions and reinforces whichever has higher $\hat{Q}_\phi$ will push probability mass toward $a' = 9$, because $7.0 > 4.8$ — even though the true value ordering is the exact opposite ($5 > 0$). The next Bellman backup then uses this newly-likely $a' = 9$ (with its inflated estimate 7.0) as part of the bootstrap target for other states, propagating the error outward through the whole value function.

Page 19 — Offline RL: A Simple Baseline (Filtered Behavior Cloning)

Before reaching for a full off-policy critic, here is the simplest possible way to use reward labels: throw away the worst trajectories and just imitate what is left.

Mechanism

1. Rank whole trajectories by total return: $r(\tau) = \sum_{(s_t, a_t) \in \tau} r(s_t, a_t)$.
2. Keep only the top-performing slice of the dataset: $\tilde{\mathcal{D}} : \{\tau \mid r(\tau) > \eta\}$, for some return threshold η .
3. Run ordinary behavior cloning (maximum likelihood) on the filtered set: $\max_\theta \sum_{(s, a) \in \tilde{\mathcal{D}}} \log \pi_\theta(a \mid s)$.

Symbol	What it means
$r(\tau)$	total return of one whole trajectory
η	the return cutoff; only trajectories scoring above this survive
$\tilde{\mathcal{D}}$	the filtered dataset, containing only high-return trajectories

Numerical Walkthrough

Suppose $\mathcal{D}$ has five trajectories with returns $r(\tau) = \{2, 8, -1, 6, 4\}$, and we set $\eta = 5$ (roughly, keep the top 40%). Then $\tilde{\mathcal{D}} = \{\tau : r(\tau) \in \{8, 6\}\}$ — only the two best trajectories survive, and behavior cloning is run on just their (s, a) pairs, ignoring the rest of the dataset entirely.

Limitation. This baseline never combines partial trajectories the way stitching does (Page 16): it can only ever be as good as the single best *whole* trajectory that happened to be recorded, and it wastes all the reward information contained in the discarded lower-return trajectories (which, per the stitching argument, might still contain a locally-excellent sub-path worth reusing).

Page 20 — Weighted Imitation Learning

Page 20 — Weighted Imitation Learning

Instead of a hard cutoff (keep this trajectory / discard that one), why not softly weight *every single transition* by how good that specific action was, using a continuous score rather than a whole-trajectory return?

Mechanism

The natural per-transition “how good was this action” score is the advantage function, recalled from earlier in the course:

$$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s).$$

Weighted imitation then reweights the log-likelihood objective by (an increasing function of) this advantage:

$$\theta \leftarrow \arg \max_{\theta} \mathbb{E}_{s,a \sim \mathcal{D}} [\log \pi_{\theta}(a \mid s) \exp(A(s, a))].$$

Symbol	What it means
$Q^\pi(s, a)$	expected return of taking a in s , then following π thereafter
$V^\pi(s)$	expected return of state s , averaged over π ’s own action choices
$A^\pi(s, a)$	how much better (or worse) a is than π ’s “typical” action in s
$\exp(A(s, a))$	converts the advantage into a positive weight > 0 for that (s, a) pair

Because $\exp(\cdot)$ is always positive and increasing, actions with a large positive advantage get an outsized weight in the log-likelihood sum, while actions with negative advantage get down-weighted toward (but never fully to) zero — so the policy is pulled toward reproducing the *better-than-typical* actions in the dataset and away from reproducing the worse ones, rather than treating every recorded action as equally worth imitating. This directly connects to Page 16’s stitching diagram: at the shared state, the branch of the graph that leads toward the rewarding state s_9 gets a large positive advantage (and hence a large weight, shown as the bold arrow in the “Adv. weights for s_3 ” box), while the branch leading to the low-reward state s_6 gets a small or negative advantage (thin arrow) — so advantage weighting is precisely the mechanism that accomplishes the “stitching” described qualitatively on Page 16.

Page 21 — Advantage-Weighted Regression (AWR)

Page 21 — Advantage-Weighted Regression (AWR)

This slide fills in *how* to actually get $V(s)$ and how to keep the exponential weight from Page 20 numerically well-behaved, turning the idea into a full, runnable algorithm. Source: Peng, Kumar, Zhang & Levine, *Advantage-Weighted Regression: Simple and Scalable Off-Policy Reinforcement Learning* (arXiv:1910.00177, 2019), as linked directly on the slide.

Mechanism

Step 1 — fit a value function by Monte Carlo regression onto the empirical (observed) return-to-go:

$$\min_{\phi} \sum_{s_t \sim \mathcal{D}} \left\| \hat{V}_{\phi}^{\pi_{\beta}}(s_t) - \sum_{t'=t}^T r(s_{t'}, a_{t'}) \right\|^2.$$

Step 2 — train the policy with an advantage-weighted log-likelihood, using a temperature α inside the exponent:

$$\max_{\theta} \mathbb{E}_{(s_t, a_t) \sim \mathcal{D}} \left[\log \pi_{\theta}(a_t | s_t) \exp \left(\frac{1}{\alpha} \left(\sum_{t'=t}^T r(s_{t'}, a_{t'}) - \hat{V}_{\phi}^{\pi_{\beta}}(s_t) \right) \right) \right].$$

Symbol	What it means
$\hat{V}_{\phi}^{\pi_{\beta}}(s_t)$	the fitted value function, an estimate of how good s_t is under the <i>behavior</i> policy π_{β}
$\sum_{t'=t}^T r(s_{t'}, a_{t'})$	the empirical (Monte Carlo) return-to-go actually observed after s_t in the data
α	a temperature hyperparameter dividing the advantage before exponentiating

The quantity inside the exponent, $\sum_{t'=t}^T r(s_{t'}, a_{t'}) - \hat{V}_{\phi}^{\pi_{\beta}}(s_t)$, is exactly a Monte Carlo estimate of the advantage $A^{\pi_{\beta}}(s_t, a_t)$ from Page 20 (observed return minus the value baseline). α exists purely for numerical stability: because $\exp(\cdot)$ grows extremely fast, dividing the advantage by α before exponentiating keeps the weights from blowing up to numbers too large for stable gradient updates — a larger α flattens (softens) the weighting, a smaller α sharpens it toward favoring only the very best actions.

Note on the slide’s return formula. The formula on the slide sums undiscounted rewards $\sum_{t'=t}^T r(s_{t'}, a_{t'})$; the original AWR paper’s more general presentation uses a discounted return, $\sum_{t'=t}^T \gamma^{t'-t} r(s_{t'}, a_{t'})$. The undiscounted version shown here is simply the finite-horizon (episodic) special case with $\gamma = 1$ and is not a mistake, just a simplification worth being aware of.

Answering the slide’s question (“what do you learn for deterministic π_{β} ?”): if the behavior policy was in fact deterministic, then for every state s_t actually visited in the data there is exactly one recorded action, so the fitted $\hat{V}_{\phi}^{\pi_{\beta}}(s_t)$ and the single observed return-to-go tend to coincide, driving the advantage estimate toward zero for every recorded transition (weight $\exp(0/\alpha) = 1$ for all of them) — AWR then degenerates toward plain behavior cloning, since there is no variation in behavior at a given state to reward-weight between.

Page 22 — Monte Carlo Estimation: Trade-offs

A short pros/cons pause before moving to a TD-based alternative on the next page.

Advantages	Disadvantages
Simple: just sum up observed rewards, no bootstrapping equations to get wrong.	Monte Carlo estimates are noisy: a single sampled trajectory's return has high variance, especially in stochastic environments or with long horizons.
Avoids ever querying or training on any out-of-distribution action (Page 18's problem never arises), because the target is built purely from rewards that were actually observed.	$\hat{A}^{\pi_\beta}$ estimates the advantage of the <i>behavior</i> policy π_β , not of the (better) policy π_θ we're actually trying to learn — so we are optimizing against a weaker yardstick than we ultimately want.

The second disadvantage is the direct motivation for the next slide: can we estimate advantages with TD updates (which bootstrap off a Q-function that tracks the *current*, improving policy π_θ) instead of Monte Carlo (which is stuck estimating the advantage of the old data-collecting policy π_β)?

Page 23 — Advantage-Weighted Actor-Critic (AWAC)

Page 23 — Advantage-Weighted Actor-Critic (AWAC)

AWAC replaces AWR's noisy, π_β -locked Monte Carlo advantage with a TD-estimated advantage that tracks the current policy π_θ — while trying to keep AWR's key safety property (never training the *policy* on out-of-distribution actions). Source: Nair, Gupta, Dalal & Levine, *AWAC: Accelerating Online Reinforcement Learning with Offline Datasets* (arXiv:2006.09359, 2020).

Mechanism

Step 1 — estimate a Q-function with a standard TD target:

$$\min_{\phi} \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} \left[\left(\hat{Q}_{\phi}^{\pi_{\theta}}(s,a) - (r + \gamma \mathbb{E}_{a' \sim \pi_{\theta}(\cdot|s')} [\hat{Q}_{\phi}^{\pi_{\theta}}(s',a')]) \right)^2 \right], \quad \text{alternative: } a' \sim \mathcal{D}.$$

Step 2 — update the policy with AWR, using this TD-based advantage instead of a Monte Carlo one:

$$\theta \leftarrow \arg \max_{\theta} \mathbb{E}_{s,a \sim \mathcal{D}} [\log \pi_{\theta}(a | s) \exp(\hat{A}^{\pi_{\theta}}(s,a))], \quad \hat{A}^{\pi_{\theta}}(s,a) = \hat{Q}_{\phi}^{\pi_{\theta}}(s,a) - \mathbb{E}_{\bar{a} \sim \pi_{\theta}(\cdot|s)} [\hat{Q}_{\phi}^{\pi_{\theta}}(s,\bar{a})].$$

Symbol	What it means
$\hat{A}^{\pi_{\theta}}(s,a)$	advantage estimated from the learned Q , using the <i>current</i> policy π_{θ} as the baseline
$\bar{a}$	a fresh action sampled from $\pi_{\theta}(\cdot s)$, used only to compute the baseline $\mathbb{E}[\hat{Q}(s,\bar{a})]$

Contrast to avoid confusing: which step actually queries OOD actions? There are *two* places an action from π_θ could be plugged into $\hat{Q}_\phi$, and they behave very differently:

- Step 2’s baseline, $\mathbb{E}_{\bar{a} \sim \pi_\theta(\cdot|s)}[\hat{Q}_\phi(s, \bar{a})]$, is subtracted off inside the advantage, but the policy’s log-likelihood term $\log \pi_\theta(a | s)$ in that same objective is always evaluated at an action *a taken from the dataset $\mathcal{D}$* — so the policy itself is never directly trained to increase probability on a self-sampled, possibly OOD action. This is the “+” marked on the slide: “still only trains policy on actions in the dataset.”
- Step 1’s TD target, however, does bootstrap through $\hat{Q}_\phi(s', a')$ with $a' \sim \pi_\theta(\cdot | s')$ — exactly the OOD query pattern flagged as dangerous on Page 18. This is the “−” marked on the slide: “queries on OOD actions.” The slide notes an alternative that avoids even this: sample $a' \sim \mathcal{D}$ instead (i.e., use the actually-recorded next action, a SARSA-style target), fully sidestepping any OOD query in the critic as well.

The “+” relative to plain AWR is that $\hat{Q}_\phi^{\pi_\theta}$ tracks the value of the policy we are currently improving, π_θ , rather than being frozen at the value of the old data-collecting policy π_β the way AWR’s Monte Carlo advantage was (Page 22’s second disadvantage) — so AWAC gets a better-targeted advantage signal, at the cost of reintroducing a (milder, critic-only) version of the OOD problem.

Page 24 — Motivating Expectile Regression

Page 24 — Motivating Expectile Regression

Both AWR and AWAC still bootstrap through the policy’s own action distribution somewhere. Can we get a TD-style value estimate that is optimistic (biased toward the best available action) *without ever evaluating Q at an action outside the dataset at all?*

Mechanism

The slide’s histogram plots $\mathbb{E}_{a \sim \pi_\beta(\cdot|s)}[Q(s, a)]$ across the range of actions actually taken by the behavior policy at state s (the data support). An ordinary squared-error (ℓ_2) fit of $V(s)$ to this distribution recovers its *mean* — the average value of a typical behavior-policy action, marked by the left dashed line. But what we actually want for $V(s)$ is closer to the value of the *best* action still contained within the data support (the right dashed line, further right / higher) — an optimistic-but-in-support estimate, without ever having to plug in an action outside that histogram (which is exactly the OOD danger from Page 18). The question posed is whether a different loss function than plain ℓ_2 can shift the fitted value from “the mean” toward “a high percentile” of this same in-support distribution.

Page 25 — Expectile Regression

Page 25 — Expectile Regression

Ordinary least-squares gives you the mean of a distribution because it penalizes positive and negative errors identically. Expectile regression breaks that symmetry, so the mini-

mizer shifts away from the mean toward a chosen upper (or lower) percentile-like statistic — the “expectile.”

Mechanism

$$\ell_2^\lambda(x) = \begin{cases} (1 - \lambda)x^2, & x < 0, \\ \lambda x^2, & \text{otherwise.} \end{cases}$$

Symbol	What it means
x	the residual (predicted value minus target, or vice versa, depending on context)
λ	the asymmetry parameter, $\lambda \in (0, 1)$, controlling which side is penalized more

When $\lambda = 0.5$, both branches have weight 0.5 and this is exactly ordinary ℓ_2 loss, whose minimizer is the mean. When $\lambda \neq 0.5$, one side of the residual is penalized more heavily than the other, so the loss is minimized not at the mean but at a value shifted toward whichever side is penalized *less* (since the fit “gives way” more easily on the cheap side and resists more on the expensive side). The plot on the slide shows this directly: for $\lambda = 0.01$ the loss curve is nearly flat for $x < 0$ and steep for $x > 0$, and for $\lambda = 0.99$ it’s the mirror image.

Numerical Verification

Take a toy distribution of $Q(s, a)$ values over four in-support actions, each equally likely: $\{2, 4, 6, 8\}$ (true mean = 5). Define the residual as in the next slide’s convention, $x = V(s) - Q(s, a)$, and numerically minimize $\mathbb{E}[\ell_2^\lambda(x)]$ over candidate values of $V(s)$ (grid search):

λ	minimizing $V(s)$
0.01	7.88 (close to the max, 8)
0.10	7.00
0.50	5.00 (exactly the mean)
0.90	3.00
0.99	2.12 (close to the min, 2)

This confirms, with actual numbers, that *small* λ pushes $V(s)$ toward the *maximum* of the in-support Q -distribution, and $\lambda = 0.5$ exactly reproduces the ordinary mean. This is the reverse of the residual convention used in most published expectile-regression write-ups (which define the residual as target-minus-prediction, $Q - V$, so that a large parameter close to 1 gives the high expectile) — but because this slide’s loss uses the residual $V(s) - Q(s, a)$ (predicted minus target, the opposite sign), the direction flips, and it is *small* λ that yields the high, near-maximum expectile. This is exactly why Page 26 specifies “using small $\lambda < 0.5$ ” to make $\hat{V}(s)$ approximate the best in-support action’s value, matching the goal illustrated by Page 24’s histogram.

Page 26 — Implicit Q-Learning (IQL)

Page 26 — Implicit Q-Learning (IQL)

This slide assembles the full offline algorithm that achieves Page 24’s goal: an optimistic, in-support value estimate, obtained entirely from TD updates, without ever plugging an out-of-distribution action into Q or V .

Mechanism

Step 1 — fit $\hat{V}(s)$ with the expectile loss, comparing it only to Q -values of actions actually in the dataset:

$$\hat{V}(s) \leftarrow \arg \min_V \mathbb{E}_{(s,a) \sim \mathcal{D}} \left[\ell_2^\lambda(V(s) - \hat{Q}(s,a)) \right], \quad \text{using small } \lambda < 0.5.$$

Step 2 — update Q with an ordinary MSE / TD loss, but bootstrapping off $\hat{V}(s')$ (a function of the *state* s' only) rather than off $Q(s', a')$ evaluated at some sampled next action:

$$\hat{Q}(s,a) \leftarrow \arg \min_Q \mathbb{E}_{(s,a,s') \sim \mathcal{D}} \left[(Q(s,a) - (r + \gamma \hat{V}(s')))^2 \right].$$

Step 3 — extract a policy with AWR, exactly as on Page 21/23:

$$\hat{\pi} \leftarrow \arg \max_{\pi} \mathbb{E}_{s,a \sim \mathcal{D}} \left[\log \pi(a | s) \exp \left(\frac{1}{\alpha} (\hat{Q}(s,a) - \hat{V}(s)) \right) \right].$$

Symbol	What it means
--------	---------------

$\hat{V}(s)$	a state-only value function, trained to be an optimistic, in-support estimate
--------------	---

$\hat{Q}(s,a)$	the Q -function, trained with ordinary TD regression, bootstrapping through $\hat{V}$
----------------	---

Why this closes the OOD loophole for good. Compare this directly with AWAC (Page 23), whose one remaining weakness was that its Q -update needed $a' \sim \pi_\theta(\cdot | s')$ (an action sampled from the current policy, potentially outside the data). Here, Step 2’s target is $r + \gamma \hat{V}(s')$ — and because $\hat{V}$ takes only a *state* as input, computing it never requires sampling or specifying *any* action at s' at all. Step 1, meanwhile, only ever evaluates $\hat{Q}(s,a)$ at pairs (s,a) literally drawn from $\mathcal{D}$. So across the whole algorithm, Q is never queried at an action it wasn’t trained on, and the “implicit” in the name refers to exactly this trick: the effect of a max over actions (needed, informally, to be optimistic) is achieved *implicitly* through the asymmetric expectile loss on in-support data, rather than through an explicit $\max_a$ or a sampled $a' \sim \pi_\theta$.